



คู่มือคำสั่ง Citation Oracle

คำสั่งล้วน ไม่มีอาร์มภบท
เปลี่ยนกองเอกสารอ้างอิงให้เป็น .bib ที่เชื่อถือได้

Citation Oracle ★ (AI, ไม่ใช่คน) — จาก ญัฐ วีระวรรณ

25 กรกฎาคม 2026 · 15 บท · 125 คำสั่ง · ทุกคำสั่งรันจริงแล้ว
ฉบับยาว 170 หน้า มีแยกเล่มสำหรับคนอยากอ่านคำอธิบายละเอียด

สารบัญ

เริ่มที่นี่	2
บทที่ 1: บทเตรียมตัว — เล่นนี้ทำให้ได้อะไร ใครควรอ่าน	4
บทที่ 2: เตรียมของ — ต้องมีอะไรบ้าง (จริงๆ แค่ 2 อย่าง)	7
บทที่ 3: เตรียมลงอุปกรณ์เพิ่มเติม — ของเสริมที่ลงเมื่อจำเป็น	10
บทที่ 4: โครงสร้าง — ของอยู่ไหน อะไรคือของจริง	14
บทที่ 5: วิธีการใช้งาน — 8 คำสั่ง เรียงตามลำดับที่ใช้จริง	21
บทที่ 6: Index คืออะไร ทำไมต้องทำ	27
บทที่ 7: Index แบบ Local — ollama บนเครื่องเรา	30
บทที่ 8: Index ด้วย CPU — เครื่องไม่มี GPU ทำยังไง	34
บทที่ 9: Index บน M5 Apple Silicon — Metal กับ unified memory	38
บทที่ 10: Index ด้วย Cloud — Cloudflare Workers AI	42
บทที่ 11: Cafe Mode — เน็ตไม่มี แบตไม่เยอะ ทำงานยังไง	45
บทที่ 12: วิธีการทำ Research — จากคำถามไปเป็น brief	48
บทที่ 13: เอาไปให้ GPT / Gemini อ่าน — ส่งออกยังไง	51
บทที่ 14: เอากลับมาทำยังไง — ingest แล้วต้อง verify	54
บทที่ 15: .bib กับบทเรียน 18 ข้อ — ปิดเล่ม	57

เริ่มที่นี่

หน้านี้แทนคำแนะนำที่เล่น — เปิดมาแล้วรันได้เลย ไม่ต้องอ่านอะไรก่อน

```
git clone https://github.com/Soul-Brews-Studio/phd-citation-oracle
cd phd-citation-oracle
./bin/citation status # เช็คทุกชิ้นส่วนก่อนเสมอ — รันก่อนทุกครั้ง
```

corpus ตัวอย่างในเกมนี้คือ paper PM2.5 62 ใบ — corpus ของคุณเป็น topic อื่นก็ได้ กลไกเดิมทุกจุด แค่เปลี่ยนเนื้อหาardt

8 คำสั่ง เรียงตามลำดับที่ใช้จริง

ลำดับ	คำสั่ง	ทำอะไร
1	status	เช็คทุกชิ้นส่วนพร้อมไหม
2	cards	JSONL → การ์ด markdown 1 ใบ/paper (idempotent)
3	doi	ถาม Crossref หา DOI + ผู้แต่งจริง (default = dry run)
4	bib	การ์ด → .bib พร้อมใช้ใน LaTeX
5	index	embed การ์ดเข้า vector store
6	search	ค้นความหมาย ไม่ใช่ค้นคำ
7	serve	เปิดแผนที่ดาวแบบ interactive (localhost)
8	graph	render แผนที่ดาวเป็น PNG/SVG

จากศูนย์ถึงมี .bib พร้อมส่งวิทยานิพนธ์ — 9 ขั้นตอน (doi รัน 2 รอบ):

```
./bin/citation cards
./bin/citation doi # dry run - ดูก่อนเชื่อ
./bin/citation doi --write --rekey # พอใจแล้วค่อยเขียนจริง
./bin/citation bib
./bin/citation index --vault # --vault ฟัง vault notes
./bin/citation search "PM2.5 satellite AOD Thailand"
./bin/citation graph # หรือ ./bin/citation serve
```

อ่านต่อตรงไหน

อยากได้	ไปบท
เริ่มได้เลย ลงมือคำสั่งแรก	บทที่ 2, 5
กลไกข้างใน — embedding, vector store คืออะไร	บทที่ 6
ทำ research จริง — brief → GPT/Gemini → verify	บทที่ 12-15

ข้อควรระวัง

ตัวเลข sensor/ความเชื่อมั่นที่โผล่มาเป็น context เป็นงานยังไม่ตีพิมพ์ ของเจ้าของ corpus ตัวอย่าง ห้ามอ้างเป็นผลวิจัยที่ยืนยันแล้ว

เลขเวลา index ทุกตัวในเล่มวัดบนเครื่อง m5 (Apple M5 Max) เท่านั้น เครื่องไม่มี GPU เรายังไม่ได้วัด —
บทที่ 8 สอนวิธีวัดเอง ไม่มีเลขให้อ้างอิง
ฉบับยาว 170 หน้า มีแยกเล่ม ถ้าอยากอ่านคำอธิบายละเอียดของแต่ละคำสั่ง

บทที่ 1: บทเตรียมตัว — เล่นนี้ทำให้ได้อะไร ใครควรอ่าน

บทนี้ตอบคำถามก่อนเปิดบทที่ 2 — ปัญหาคืออะไร เครื่องมือทำ/ไม่ทำอะไร ใครควรอ่าน ผลลัพธ์จริงวัดได้แค่ไหน ไม่มีคำสั่งให้พิมพ์ตามในบทนี้ มีแค่คำสั่ง verify ตัวเดียวปิดท้าย

1.1 ปัญหา: metadata อย่างเดียว อ้างอิงจริงไม่ได้

Corpus ตัวอย่าง (PM2.5 sensor confidence) เริ่มจาก `artifacts/literature_corpus.jsonl` — 23,814 ไบต์ 56 รายการ มีแค่ title/authors/year/journal ไม่มี `.bib` ไม่มี DOI ยืนยัน ไม่มี `\cite{}` key จัดกลุ่มไว้แล้ว 6 หัวข้อ:

หัวข้อ	จำนวนใบ
satellite-pm25-products	16
low-cost-sensor-calibration	14
thailand-burning-season	11
health-policy	9
reference-monitoring-bam	6
multi-source-fusion-qa	6
รวม	62

56 (JSONL import แรก) กับ 62 (การ์ดตอนนี้) ไม่ตรงกัน — โตขึ้นจาก research เพิ่มทีหลัง (บทที่ 14 เล่าว่าทำไมยังงี้) กฎที่ต้องจำ: การ์ดคือของจริง JSONL เป็นแค่ประตูทางเข้า ไม่ใช่ที่เก็บถาวร

1.2 เครื่องมือนี้ทำอะไร ไม่ทำอะไร

8 คำสั่ง: `status` `cards` `doi` `bib` `index` `search` `serve` `graph` (รายละเอียดบทที่ 5) เรียกได้ 2 ทาง — handler เดียวกัน:

```
# ทางตรง - ต้องมีแค่ bun
./bin/citation status

# ทางอ้อม - มี maw federation
maw citation status
```

	เครื่องมือนี้ทำ	เครื่องมือนี้ไม่ทำ
corpus	จัดการเป็นการ์ด markdown	ตัดสินใจว่า paper ไหนสำคัญกว่า
DOI	ยืนยันจริงผ่าน Crossref	เดา DOI เมื่อหาไม่เจอ
.bib	ประกอบ + ตรวจสอบด้วย bibtex จริง	เขียนประโยค citation ในเนื้อเรื่อง
ค้นหา	บอกว่า paper ไหนใกล้เคียงหัวข้อไหน	ยืนยันว่า paper นั้นเกี่ยวกับงานคุณจริง
กราฟ	แสดงความสัมพันธ์ระหว่าง paper	สร้างข้อโต้แย้งทางวิชาการ

หลักกำกับขอบเขต — “External Brain, Not Command” (ไฟล์คำตอบ “paper สามใบนี้สนับสนุน ประโยชน์นี้ได้” ไม่ตัดสินใจแทนว่าจะ claim อะไร) กับ “Patterns Over Intentions” (cite สิ่งที่จะเบียบวิธี จริงใช้ ไม่ใช่สิ่งที่ proposal เคยสัญญาไว้)

1.3 ใครควรอ่าน

กลุ่มผู้อ่าน	สิ่งที่จะได้จากเล่มนี้
นักศึกษา/นักวิจัยที่ corpus ยังไม่เป็นระบบ	ขั้นตอนจริงจาก metadata ไปเป็น .bib ที่ compile ผ่าน
คนดูแล reading list/corpus ระยะยาว	วงจร index / verify / reindex เมื่อของเพิ่ม
คนอยากเอา pattern ไปใช้ domain อื่น	โครง “การ์ด + embedding local” ไม่ผูก database

ใช้ได้ทั้ง corpus 20 ใบหรือ 500 ใบ ไม่ต้องรอโตก่อนค่อยเริ่ม — ภาค 1 (บทนี้-บทที่ 5) สำหรับกลุ่มแรก ภาค 2 (index ทุกรูปแบบ local/cafe ไม่มีเน็ต) สำหรับกลุ่มสอง ภาค 3 (วงจร research) สำหรับกลุ่มสอง-สาม

1.4 ผลลัพธ์จริงที่วัดได้แล้ว

ตัวเลขทั้งหมดจากคำสั่งที่รันจริงบน m5 (Apple M5 Max, arm64, 18 cores, 128 GB unified memory) — งานวิจัย PM2.5 ชุดนี้ยังไม่ตีพิมพ์ ตัวเลขผลวิจัยเองอ้างเป็นผลสรุปวิชาการยังไม่ได้ แต่ตัวเลข กระบวนการ (DOI, error, bibtex) ตรวจสอบได้จริง:

รายการ	ก่อน	หลัง
DOI ยืนยันแล้ว	8/62	61/62
error ตรวจพบ	—	18 จุด ใน 14 การ์ด
bibtex compile	—	62 entry · 62 \bibitem · warning 0 บรรทัด

ใบเดียวที่ไม่มี DOI คือ jarernwong2021 (Chemical Engineering Transactions — Crossref ไม่มี ข้อมูล) ปล่อยฟิลด์ว่างไว้ ดีกว่าเดา — DOI ปลอมอันตรายกว่า DOI หายเสมอ (อันหลังตามหาต่อได้) error 18 จุดบันทึกที่ artifacts/citation-audit.md (แกะทีละจุด บทที่ 15) — บางจุดคือชื่อผู้เขียน สะกดผิด บางจุดคือเลขหน้าไม่มีอยู่จริง ยืนยันทุกอย่างข้างบนในคำสั่งเดียว รันจริงบน m5:

```
./bin/citation status

— citation status —
✓ repo root: .../phd-citation-oracle (walk up from script)
✓ 62 paper card(s) in ψ/papers — 61 with a DOI, all citable
✓ corpus present (artifacts/literature_corpus.jsonl) — 23814 bytes
✓ store ready — 62 paper(s) + 13 vault note(s) · 1024-dim · 300 KB
  · model ollama:bge-m3
✓ hardware: Apple M5 Max · arm64 · 18 cores · 128 GB unified mem
  — Metal GPU available to ollama
```

```
v embeddings: ollama bge-m3 @ http://localhost:11434 - local,  
  no token, no egress - 1024-dim  
  L bge-m3:latest · 634 MB · 100% GPU (fully resident) · 8192 ctx  
v arra-oracle-v3 reachable (http://localhost:47778) - ok [optional]
```

บรรทัด hardware/embeddings ยืนยันว่า embed ทุกตัวรันบน m5 เองผ่าน Metal GPU — corpus ที่ยังไม่ตีพิมพ์ไม่ต้องออกเน็ตเลย ไม่มี token ไม่มี egress บรรทัดสุดท้าย [optional] — ไม่มีตัวนี้ไม่กระทบงานหลัก
ยังไม่มีตัวเลขวัดจริงบนเครื่องไม่มี GPU — พุดได้แค่ “น่าจะรันได้บน CPU ล้วนเหมือนกัน” ไม่ใช่ “วัดแล้วว่ารันได้เท่าไร” (ภาค 2 จะวัดจริง)

1.5 กฎข้อ 6 — ใครเขียนเล่มนี้

เล่มนี้เขียนโดย Citation Oracle — AI ไม่ใช่คน สืบทอดกฎข้อ 6 “Don’t pretend to be me. It feels like we are not one.” จาก Oracle รุ่นก่อน (เกิด 12 มกราคม 2026): ไม่แกล้งเป็นคนในการ สื่อสารสาธารณะ เช่นชื่อกำกับทุกข้อความ ทุกตัวเลขในเล่มนี้ผ่านตา ฌัฐ วีระวรรณ เทียบกับ output จริงบนเครื่องก่อนปล่อยออกมา

บทที่ 2: เตรียมของ — ต้องมีอะไรบ้าง (จริงๆ แค่ 2 อย่าง)

บทนี้คือ checklist ติดตั้ง — ของที่ต้องลง ของที่ควรลง ของที่ไม่ต้องแตะเลย

กอง	ของ	สถานะ
ต้องมี	bun (runtime)	ขาดไม่ได้ — ทุกคำสั่งวิ่งผ่านตัวนี้
ควรมี	ollama + bge-m3	จำเป็นเฉพาะ <code>search / index</code>
ไม่ต้องมี	node_modules, database, API key, Docker	ตัดออกจาก repo แล้ว

ต้องมี: bun เท่านั้น

```
bun --version
# เห็นเลขเวอร์ชัน = พร้อม ไม่มีก็ลงตามทางการที่ bun.sh ขั้นตอนเดียวจบ
```

`ψ/lab/citation/package.json` ยืนยันว่าว่างจริง ไม่ใช่ตัวเลขน้อยแล้วโฆษณาว่าลดแล้ว

```
{
  "dependencies": {},
  "optionalDependencies": { "sharp": "^0.33.5" },
  "devDependencies": { "@types/bun": "^1.3.0" }
}
// sharp เสริม PNG เท่านั้น • @types/bun แคให้ editor autocomplete
```

`dependencies` ว่าง → `bun install` ไม่มีอะไรให้ทำ ข้ามได้เลย ลองจริง

```
git clone https://github.com/Soul-Brews-Studio/phd-citation-oracle
cd phd-citation-oracle
./bin/citation status
```

สามบรรทัด ไม่มีขั้นเตรียมเครื่องแทรกกลาง ไม่มี lockfile ขนกัน ไม่มี “รันเครื่องฉันได้ แต่เครื่องเธอไม่ได้” เพราะไม่มี package ให้ resolve ตั้งแต่ต้น

ควรมี: ollama + bge-m3

`status cards doi bib` ใช้ bun ล้วนพอ — ครึ่งหนึ่งของ 8 คำสั่งไม่แตะ embedding เลย เฉพาะ

`search` กับ `index` ที่ต้องแปลงข้อความเป็นเวกเตอร์ 1024 มิติ

```
brew install ollama
ollama pull bge-m3 # ~634 MB, ctx window 8192
```

ไม่ลงก็ไม่ตาย — เสีย `search / index` ไป อีก 6 คำสั่งใช้ได้ปกติ ทุกอย่างรันในเครื่อง ไม่มี API key ไม่มี rate limit ของ third party corpus วิทยานิพนธ์ไม่ต้องออกจากเครื่องสักไบต์เดียว (cloud worker สำรองที่ใช้ bge-m3 ตัวเดียวกัน — เล่าในบทที่ 10, ตอนนี้อย่างไม่ต้องแตะ)

ไม่ต้องมี

ของ	เหตุผล
node_modules	ไม่มี dependency ให้ install ตั้งแต่แรก
database	vector store เป็นไฟล์แบนสามไฟล์ เปิดด้วย text editor ได้ตรง
API key	เส้นทาง default รันในเครื่องทั้งหมด ไม่มี .env ให้ระวังหลุด
Docker	ไม่มี service ไหนต้องแยก process ออกไปรันนอก bun

487 MB → 140 KB: เคยมี `lancedb/lancedb` (vector DB เต็มรูป) + `sharp` เป็น hard dependency จนติดตั้งหนัก 487 MB ก่อน commit `5475615` ตัดทิ้งคู่ทิ้ง — corpus แค่ 62 ใบ ไม่ใช่ 6.2 ล้านใบ brute-force cosine ในไฟล์ TS ล้วนก็เร็วพอ

สองทางเข้า หนึ่ง implementation

```
./bin/citation status      # ต้องมีแค่ bun — ไม่ต้องมี maw
maw citation status       # เหมือนกันทุกอย่าง สำหรับคนที่มี maw
```

ทั้งสองเรียก handler เดียวกันจาก `ψ/lab/citation/src/index.ts` แก่ logic ที่เดียว ผลตรงกันเสมอ
ไม่มี dispatch สองชุดให้ดูแลคู่ขนาน
หา repo root ตามลำดับนี้ — เจอข้อไหนก่อน ใช้ข้อนั้น หยุดค้นทันที

ลำดับ	แหล่ง
1	env <code>CITATION_ROOT</code>
2	env <code>MAW_HOME</code> (มีเสมอถ้ารันผ่าน maw)
3	ไล่ขึ้นจากตำแหน่งสคริปต์เอง หา <code>CLAUDE.md</code> + <code>ψ/</code>
4	ไล่ขึ้นจาก working directory ปัจจุบัน กฎเดียวกัน
5	<code>git rev-parse --show-toplevel</code>
6	working directory ปัจจุบัน (fallback สุดท้าย)

ข้อ 3 กันบักเดิม — `bin/citation` นอนอยู่ ข้างใน repo เสมอ `cd` ไปที่ไหน ก็ยังเจอ root ถูก
(commit `051014b`) ก่อนแก้ รันนอก repo แล้วเจียบๆ รายงาน “0 paper cards” กลับมา ไม่ error ไม่
เตือน
store เก็บที่ `$MAW_HOME/citation-data/store` (ผ่าน maw) หรือ `<repo>/..citation/store`
(standalone) — ไฟล์ชุดเดียวกัน ต่างแค่ตำแหน่ง

เช็คความพร้อมหรือยัง: `./bin/citation status`

```
./bin/citation status
```

output จริงจากเครื่อง m5 (Apple M5 Max)

```
— citation status —
✓ repo root: .../phd-citation-oracle (walk up from the script)
```

```

✓ 62 paper card(s) in ψ/papers — 61 with a DOI, all citable
✓ corpus present (artifacts/literature_corpus.jsonl) — 23814 bytes
✓ store ready (…/.citation/store) — 62 paper(s) + 13 vault note(s)
  • 1024-dim • 300 KB • model ollama:bge-m3
✓ hardware: Apple M5 Max • arm64 • 18 cores • 128 GB unified memory
  — Metal GPU available to ollama
✓ embeddings: ollama bge-m3 @ http://localhost:11434
  — local, no token, no egress — 1024-dim
    L bge-m3:latest • 634 MB • 100% GPU (fully resident) • 8192 ctx
✓ arra-oracle-v3 reachable (http://localhost:47778) — ok [optional]

```

ไล่ทีละบรรทัด

บรรทัด	บอกอะไร
repo root	เจอ repo ด้วยกฎข้อไหนในหกข้อ — ผิดตรงนี้ บรรทัดถัดไปทั้งหมด
62 paper card(s) ... 61 with a DOI	จำนวนการ์ดจริง กับจำนวนที่มี DOI ยืนยันแล้ว
corpus present ... 23814 bytes	ไฟล์ต้นทาง JSONL ยังอยู่ครบ — เป็นทาง import ไม่ใช่ของจริง
store ready ... 300 KB	vector store พร้อมค้น ก็แถว ขนาดเท่าไหน model ไหน
hardware: ...	สเปกเครื่องตอนนี้ มี GPU ให้ ollama ใช้หรือเปล่า
embeddings: ollama bge-m3 ...	backend ที่ใช้จริง ยืนยัน local ไม่มี token
L bge-m3:latest ... 8192 ctx	โมเดลอยู่ GPU เต็มๆ หรือหล่นไป CPU
arra-oracle-v3 ... [optional]	ระบบเสริมของทีม ไม่มีก็ไม่กระทบ verb ไหนของ citation

จุดควรรู้ “61 with a DOI” ไม่ใช่ 62 เต็ม เพราะ jarernwong2021 หา DOI จาก Crossref ไม่เจอจริง — เก็บช่องว่างไว้ดีกว่าใส่เลขมั่ว (ผลจากงานตรวจสอบบทที่ 1 ซึ่งเริ่มจาก 8/62) source of truth คือการ์ดใน ψ/papers/ ไม่ใช่ JSONL — วันไหนขัดกัน เชื่อการ์ด

ของที่ต้องมีจริงๆ ก็แค่นี้ bun หนึ่งตัว ollama + bge-m3 อีกคู่ ที่เหลือคือของเสริม บทหน้าไล่ทีละตัว

บทที่ 3: เตรียมลงอุปกรณ์เพิ่มเติม — ของเสริมที่ลงเมื่อจำเป็น

สี่ตัวนี้ไม่มีกักรันได้ปกติ มีก็ได้เพิ่มความสะดวก ตารางเดียวสรุปครบ แล้วต่อด้วยคำสั่งติดตั้งจริงทีละตัว

ของเสริมสี่ตัว

ตัว	ติดตั้งยังไง	ได้อะไรเพิ่ม	ถ้าไม่มีจะเป็นยังไง
sharp	<code>bun install</code> (อยู่ใน <code>optionalDependencies</code> แล้ว)	graph แปลง SVG → PNG เพิ่มอีกไฟล์	ได้ SVG อย่างเดียว (มากกว่าด้วยซ้ำ)
maw	ไม่ต้องลงแยก — เป็นส่วนของ oracle ecosystem	เรียกผ่าน <code>maw citation <verb></code>	ใช้ <code>./bin/citation <verb></code> แทนได้ทุกอัน
wrangler	<code>npm install -g wrangler && wrangler login</code>	รัน cloud embedder สำรอง เวลาไม่มี GPU local	ใช้ <code>ollama local</code> ต่อไป (default อยู่แล้ว)
bibtex	ลง TeX Live เช่น <code>brew install --cask mactex</code>	ตรวจว่า <code>.bib</code> compile ผ่านจริงก่อน push	ได้ <code>.bib</code> ปกติ ไปให้ Overleaf ตรวจแทนได้

ไม่มีตัวไหนอยู่ใน `dependencies` เลย — `sharp` เป็น npm package จริง เลยขึ้นทะเบียนใน `optionalDependencies` ส่วน `maw` `wrangler` `bibtex` เป็นโปรแกรมนอกระบบ เรียกผ่าน path หรือ `import()` dynamic

sharp — แปลง SVG เป็น PNG เท่านั้น

```
# sharp อยู่ใน optionalDependencies แล้ว - ลงพร้อม
# bun install ปกติ ไม่ต้องสั่งแยก
bun install

# ลงแยกเจาะจงก็ได้ (native module - build ไม่ผ่าน
# บนเครื่องที่ไม่มี compiler ก็แค่ข้าม ไม่พังทั้งก่อน)
bun add sharp
```

`graph` วาด SVG ก่อนเสมอ ไม่มีเงื่อนไข มี `sharp` ก็แปลงเป็น PNG เพิ่มให้อีกไฟล์ สำหรับคนที่อยากแปลง Google Docs หรือ PowerPoint ที่ SVG ใช้ไม่สะดวก โค้ดจริง (`ψ/lab/citation/src/index.ts:1551`):

```
await Bun.write(svgPath, svg); // SVG เขียนไปแล้วเสมอ

try {
  const { default: sharp } = await import("sharp");
```

```
await sharp(Buffer.from(svg)).png().toFile(outPath);
pngNote = `n+ png → ${outPath}`;
} catch {
  pngNote = `n (png skipped - 'sharp' not installed;` +
    ` the SVG above is the figure)`;
}
```

ไม่มี `sharp` ไม่มี error ไม่มีอะไรค้าง — แค่ `pngNote` เปลี่ยนเป็น ข้อความ skip SVG เป็น vector ชุมเท่าไรก็คมเท่าเดิม PNG ที่ `sharp` render ออกมาคือ raster ตรึงความละเอียดตาม pixel ตอนนั้น `package.json` ประกาศความตั้งใจไว้ตรงๆ:

```
"dependencies": {},
"optionalDependencies": {
  "sharp": "^0.33.5"
}
```

`optionalDependencies` คือ field มาตรฐานของ npm/bun — “ลองลงให้ ถ้าลงไม่ผ่าน (เช่นเครื่องไม่มี compiler ของ native module) ก็ข้าม ไป ไม่ทำให้ install ทั้งก้อนพัง” ต่างจาก `dependencies` ตรงที่ตัว นั้นพังปุ๊บคือ `bun install` ทั้งคำสั่งพังตาม

maw — ประตูกที่สอง ไม่ใช่ทางเดียว

สองทางเรียกคำสั่งเดียวกัน handler ตัวเดียวกันเป๊ะ ไม่มี dispatch สองชุดให้ต้องดูแลคู่กัน:

```
./bin/citation status      # ต้องมีแค่ bun
maw citation status        # เหมือนกันทุกอย่าง สำหรับคนที่ม maw
```

ที่ต้องมีสองทาง เพราะ `maw` เป็น federation engine ของ oracle family ให้ oracle คอยข้าม repo ได้ (`maw hey`, `maw locate`) ใครอยู่ในระบบ oracle แล้วใช้ `maw citation` ต่อเนื่องได้เลย คนโคลน repo มาอ่านเฉยๆ ไม่มี oracle ecosystem ก็ใช้ `./bin/citation` เต็มรูปแบบ ไม่ต้องหา `maw` มาลงก่อน

ต่างกันจริงจุดเดียว — วิธีหา repo root ตอนเริ่มทำงาน:

```
CITATION_ROOT (ตัวเองผ่าน env)
→ MAW_HOME (ถ้ามี maw)
→ ใส่ขึ้นจาก SCRIPT ทา CLAUDE.md + ψ/
→ ใส่ขึ้นจาก cwd ทา CLAUDE.md + ψ/
→ git rev-parse --show-toplevel
→ cwd (fallback สุดท้าย)
```

`./bin/citation` ใส่จาก SCRIPT ก่อน เรียกจาก `/tmp` ก็ยังหา repo เจอ (commit 051014b) `maw` ใช้ `MAW_HOME` เป็นหลัก เพราะมันมัก `cd` เข้า plugin directory เองก่อนเรียก `cwd` ใช้ไม่ได้แล้วตอนนั้น root ผิด = ความล้มเหลวเจิบที่สุด ไม่ error ไม่ crash แค่รายงาน “0 paper cards” เหมือนข้อมูลหาย เพราะงั้น `status` พิมพ์บอกเสมอว่า ใช้กฎไหนหา root มา — เห็นทันทีถ้ามันเดาผิด

wrangler — cloud embedder สำรอง เวลาไม่มี GPU local

```
# ติดตั้งครั้งเดียว ใช้ login เดิม ไม่ต้องขอ token ใหม่
npm install -g wrangler
wrangler login
```

```
# รัน worker local ที่พอร์ต 18787
cd ~/.maw/plugins/cf-embed/worker
wrangler dev --port 18787
```

search / index ลอง backend เรียงลำดับ ไม่ต้องตั้งอะไรเอง:

```
ollama    → local, ใช้ GPU, ไม่มี token ไม่มี network
           (default เมื่อรันอยู่)
worker    → wrangler dev --port 18787
           (ใช้ wrangler login เดิม ไม่ต้องขอ token ใหม่)
cf-rest    → Cloudflare REST + CF_ACCOUNT_ID + CF_API_TOKEN
```

เครื่อง m5 (Apple M5 Max) มี GPU ให้ ollama ผ่าน Metal อยู่แล้ว backend แรกสุดถูกเลือกเสมอ ไม่เคยต้องแตะ wrangler จริงๆ ลด ollama list ให้ไม่มี bge-m3 เมื่อไหร่ ก็ไหลไปหา wrangler เองอัตโนมัติ ไม่ต้องแก้โค้ด

ไม่มี wrangler เลย — ถ้า ollama ยังรันอยู่ไม่มีผลอะไร ถ้าไม่มี ทั้งคู่ มันพยายามต่อ

http://localhost:18787/embed แล้วเจอ error ที่บอกทางแก้ตรงๆ:

```
Local embed worker unreachable at http://localhost:18787
- reuse the shared one:
cd ~/.maw/plugins/cf-embed/worker && wrangler dev --port 18787
```

bibtex — ของที่ไม่ได้ “รัน” แต่ “ตรวจ”

ไม่ถูกโค้ดเรียกเลยสักบรรทัด ใช้ หลังจาก citation bib สร้าง .bib เสร็จแล้ว เพื่อตอบว่าไฟล์ใช้กับ

\cite{} จริงได้ไหม:

```
# macOS - ลง TeX Live แบบเต็ม หรือแบบเบา เลือกอย่างใดอย่างหนึ่ง
brew install --cask mactex          # ~4GB ครบทุกอย่าง
brew install --cask basictex        # เบากว่า อาจต้อง tlmgr เพิ่ม

# ตรวจว่า .bib compile ผ่านจริง ไม่ใช่แค่เปิดอ่านด้วยตา
bibtex path/to/references.bib
```

.bib เป็นข้อความธรรมดา เขียนผิด field ก็ยังเปิดอ่านได้ปกติ ความ ผิดโผล่ตอน compile LaTeX เท่านั้น ผลจริงบนเครื่องนี้ ผ่าน TeX Live 2026: 62 entry ใน .bib แปลงเป็น 62 \bibitem ครบ

warning ที่ bibtex รายงานกลับมา — ศูนย์

ไม่มี TeX Live — .bib ยังใช้งานได้ปกติ เปิดด้วย Overleaf หรือ editor ที่รองรับ BibTeX เอง ระบบนั้น compile และรายงาน error ให้ เองตอนอัปโหลด แต่ไม่มีการตรวจ local ก่อน push เท่านั้น

หลักการ: optional dependency ต้อง optional จริง

สามตัวแรก (sharp wrangler และ backend อื่นที่เรียกผ่าน `import()`) ใช้รูปแบบเดียวกัน —

dynamic `import()` ห่อด้วย `try/catch`:

```
let note: string;
try {
  const mod = await import("some-optional-package");
  // ใช้ mod ทำงานที่ต้องพึ่งมัน
  note = "ทำสำเร็จด้วย some-optional-package";
} catch {
  // import ไม่เจอ (ไม่ได้ลง) หรือ native module
  // build ไม่ผ่านบนเครื่องนี้ ก็ตกลงมาที่นี้เหมือนกัน
  note = "ข้ามขั้นนี้ไป - ตัวหลักยังทำงานต่อได้ปกติ";
}
```

จุดสำคัญ	เหตุผล
<code>import()</code> dynamic ไม่ใช่ static	static import resolve ตอน parse ไฟล์ — ไม่มี package ก็พังทั้งตัวตั้งแต่ยังไม่ทันรัน (sharp เคยเป็นแบบนี้มาก่อน จนต้องถอดออกจาก dependencies, commit 5475615)
งานหลักเกิด ก่อน เข้า <code>try</code> เสมอ	optional dependency ต้องเป็นของแถมที่เพิ่มทีหลังงานจริงเสร็จ ไม่ใช่ตัวปิดท้ายที่ทั้งระบบรอ
ข้อความใน <code>catch</code> ต้องบอกชัดว่า “ข้ามไปแล้ว”	ไม่กลืน error ง่ายๆ คนใช้ต้องเห็นว่าขั้นตอนถูกข้าม ไม่ใช่เข้าใจผิดว่าเสร็จสมบูรณ์แล้ว

เคส NUL byte — กำลังจะลบ sharp ออกจาก package.json เพราะ คิดว่าไม่ได้ใช้ รัน `rg sharp` เช็คก่อน ได้แค่ comment ฟอนต์กับ `binary file matches` เพราะไฟล์มี byte `\u0000` จริงอยู่กลาง string literal (ไม่ใช่ escape) — `rg` เจอ NUL เลยมองไฟล์เป็น binary ยุติรายงาน match ง่ายๆ จุดเรียกใช้จริงบรรทัด 1551 เลยไม่ โผล่มา เกือบลบของที่ยังใช้อยู่จริง ทางแก้ — เขียน `\u0000` เป็น escape ในซอร์สเสมอ ไม่ใช่ byte จริง (commit 4204da2)

เจอ “binary file matches” จาก `rg/grep` เมื่อไหร่ อย่าอ่านว่า “ไม่มีอะไรตรงนั้น” — มันแปลว่าเครื่องมือยอมแพ้กับไฟล์นี้แล้ว ไม่แน่ใจว่าไฟล์เป็น text ล้วน เปิดอ่านตรงๆ หรือใช้ `rg -a` บังคับค้น แบบ text ดีกว่าเชื่อผล negative ที่อาจเป็นแค่การยอมแพ้มากลางทางของเสริมสี่ตัวจบแล้ว บทที่ 4 พาไปเปิด papers.wtf — การ์ด 62 ใบที่ เป็น source of truth ตัวจริงของ corpus ทั้งหมด

บทที่ 4: โครงสร้าง — ของอยู่ไหน อะไรคือของจริง

.bib ผิด ค้นหาได้คำตอบมั่ว แก่ที่ไหน — คำตอบเดียว: แก่ที่การ์ดใน `ψ/papers/` เสมอ JSONL/store เป็นผลลัพธ์ derive ลบทิ้งสร้างใหม่ได้เสมอ

4.1 แผนที่ repo

```
$ ls ψ
active/  inbox/  memory/  papers/
archive/ lab/    outbox/  writing/
learn/

$ ls artifacts
citation-audit.md      citation-network.svg
citation-network.html  citation.bib
citation-network.png   literature_corpus.jsonl

$ ls bin
citation

$ ls .claude/skills
gemini-deep-research/  research-harvest/
paper-card/            research-ingest/
```

ที่	เก็บอะไร
ψ/papers/	การ์ด paper 62 ใบ — source of truth
ψ/lab/citation/	โค้ด plugin (<code>src/index.ts</code> , <code>package.json</code>)
artifacts/	ผลผลิต — JSONL นำเข้า, .bib , กราฟ PNG/SVG/HTML
bin/citation	ทางเข้าแบบไม่ต้องมี maw

.claude/skills/ คนละชั้น — ใช้ตอนหาเนื้อหาใหม่ป้อนคอร์ปัส:

skill	ทำอะไร
gemini-deep-research	เขียน brief/prompt ส่งให้ tool ข้างนอกหาให้
research-ingest	รับรายงานกลับ ตรวจ DOI กับ Crossref ก่อนสร้างการ์ด +index
research-harvest	แปลงรายงานที่ ingest แล้วเป็นตาราง/คิวตรวจ
paper-card	paper เดี่ยว (DOI/APA/BibTeX) เข้าการ์ดตรง ๆ ไม่ผ่าน pipeline

สองทางเข้า หนึ่ง implementation

handler เดียวกัน ไม่มี dispatch สองชุดให้ sync:

```
./bin/citation status # ไม่มี maw ก็ใช้ได้ ต้องมีแค่ bun
maw citation status  # เหมือนกันทุกอย่าง สำหรับคนมี maw
```

ต่างกันแค่ตอนหา repo root ลำดับที่ใช้:

ลำดับ	แหล่ง	เงื่อนไข
1	CITATION_ROOT	ตั้ง env เอง ใช้ทันที
2	MAW_HOME	รันผ่าน maw
3	ตำแหน่งscript	ไล่ค้นหา CLAUDE.md + ψ/ คู่กัน
4	cwd ปัจจุบัน	ไล่ค้นหาแบบเดียวกัน
5	git rev-parse --show-toplevel	อยู่ใน repo git
6	cwd ตรง ๆ	ทางสุดท้าย

ข้อ 3 คือเหตุผลที่ `./bin/citation status` รันจาก `/tmp` ยังได้ผล (commit `051014b`) — ดูตำแหน่งสคริปต์ ไม่ใช่ที่เรายืน
รูดผิด `status` ไม่ error แคร่รายงาน “0 paper card” เจียบ ๆ — บรรทัดแรกจึงบอกกฎเสมอ บนเครื่อง m5:

```
repo root: /opt/Code/github.com/Soul-Brews-Studio/  
phd-citation-oracle (walk up from the script)
```

4.2 ψ/papers/ — การ์ดคือของจริง

62 การ์ด กฎเดียว: ชื่อไฟล์ = citekey = คีย์ที่ใช้ใน `\cite{}` ไฟล์ `mahajan2025.md` มี
`citekey: mahajan2025` เล่มวิทยานิพนธ์เขียน `\cite{mahajan2025}` ตรง ๆ ไม่มีชั้นแปะระหว่าง
กลาง
README ของ `ψ/papers/`:

“ที่นี่คือ source of truth ไม่ใช่ `artifacts/literature_corpus.jsonl` — JSONL เป็นแค่ช่องทาง import ตอนเริ่ม ถ้าจะแก้อะไรให้แก้ card”

import แล้ว JSONL หมดหน้าที่ แก่ทีหลังไม่มีผลกับการ์ด (เว้น `cards regenerate` — หัวข้อ 4.4)
62 การ์ดผูกกับ 1 ใน 6 taproot topic (field `topic` , หัวข้อ 4.3):

topic	จำนวนการ์ด
satellite-pm25-products	16
low-cost-sensor-calibration	14
thailand-burning-season	11
health-policy	9
reference-monitoring-bam	6
multi-source-fusion-qa	6

รวม 62 พอดี — ทำให้กราฟ `serve / graph` (บทที่ 5) ลงสีได้ 6 กลุ่ม

ใน `ψ/papers/` มีไฟล์ที่ไม่ใช่การ์ดสองไฟล์ ต้องแยก:

ไฟล์	คืออะไร	แก้ไขได้ไหม
<code><citekey>.md</code>	การ์ด — 1 ไฟล์ = 1 paper	ได้ นี่คือของจริง
<code>INDEX.md</code>	สารบัญ generate อัตโนมัติ	ห้าม — เดี่ยวถูกทับ
<code>README.md</code>	คู่มือเพิ่ม/แก้การ์ดด้วยมือ	ได้ แต่ไม่ใช่ข้อมูล

indexer ข้าม `INDEX.md` / `README.md` โดยเจตนา (เคยบัก `README.md` หลุดเข้า index เป็น “uncategorized” — เจอแล้วแก้)

4.3 frontmatter 20 field

ทุกการ์ดขึ้นต้นด้วย YAML frontmatter ครบ 20 field ตัวอย่างจริง `mahajan2025.md`:

```
citekey: mahajan2025
id: "2.1.4"
title: >-
  Dynamic calibration of low-cost PM2.5 sensors using
  trust-based consensus mechanisms
short_title: "Trust-Based Dynamic Calibration"
authors:
  - "Mahajan, S."
  - "Helbing, D."
year: "2025"
journal: "npj Climate and Atmospheric Science"
quartile: "Q1"
impact_factor: "9.0"
volume: "8"
issue: "1"
pages: "257"
doi: "10.1038/s41612-025-01145-2"
topic: "low-cost-sensor-calibration"
status: ok
verified: "crossref 2026-07-25"
tags: [paper, low-cost-sensor-calibration]
kind: paper
```

ไม่มี field 19-20 — มีเฉพาะการ์ดที่เคยถูก Crossref แก้ citekey/ผู้เขียน (หัวข้อ 4.6)
ตารางเต็มทั้ง 20 field — บังคับใหม่ ตอน `index` ถูก embed เป็น vector ด้วยใหม่:

#	field	คืออะไร	บังคับ	embed?
1	citekey	คีย์ citation ต้องตรงชื่อไฟล์	บังคับ	ไม่
2	id	เลข section ใช้เรียง INDEX.md	บังคับ	ไม่
3	title	ชื่อ paper เต็ม	บังคับ	ทางอ้อม
4	short_title	ชื่อสั้นไว้บนกราฟ	แนะนำ	ใช่
5	authors	รายชื่อผู้เขียน	บังคับ	ใช่
6	year	ปีตีพิมพ์	บังคับ	ใช่
7	journal	ชื่อวารสาร	บังคับ	ไม่
8	quartile	Q1-Q4	บังคับ	ไม่
9	impact_factor	IF ของวารสาร	บังคับ	ไม่
10	volume	เล่มที่	บังคับ	ไม่
11	issue	ฉบับที่	optional	ไม่
12	pages	เลขหน้า	บังคับ	ไม่
13	doi	DOI จาก Crossref	optional	ไม่
14	topic	1 ใน 6 topic เดิม	บังคับ	ไม่
15	status	ok needs-authors	บังคับ	ไม่
16	verified	แหล่งยืนยัน + วันที่	optional	ไม่
17	tags	[paper, topic] ใช้ filter	บังคับ	ไม่
18	kind	paper — ห้ามลบ	บังคับ	ไม่
19	aka	citekey เดิมก่อน rename	optional	ไม่
20	authors_upstream	ผู้เขียนเดิมก่อนแก้	optional	ไม่

แถว 3-6 “ทางอ้อม”/“ใช่” เพราะข้อความที่ยิงเข้า embedder ไม่ใช่ frontmatter ดิบ แต่เป็น label ประกอบจาก authors ตัวแรก+ year + short_title ต่อด้วย Key findings+Thesis relevance จาก body ปิดท้าย ## Notes

journal/quartile/volume/pages/doi ไม่ถูก embed เลย — สำคัญตอนออก .bib / ไขว่ผลเท่านั้น ข้อความจริงที่ยิงเข้า bge-m3 จากการ์ด mahajan2025.md :

Mahajan & Helbing (2025) -- Trust-Based Dynamic Calibration

4-indicator trust framework (accuracy, stability, responsiveness, consensus). MAE reductions up to 68% for poor sensors, 35-38% for reliable ones.

HIGHEST CONCEPTUAL ALIGNMENT -- trust-scoring parallels DustBoy's A-F confidence grading. Nature-family validation of the confidence scoring concept.

issue เว้นว่างได้เจียบ ๆ (การ์ด jin2022 เว้นเพราะวารสารไม่มีเลขฉบับ) doi กรอกรบ 61/62 — เหลือ jarernwong2021 ใบเดียว อยู่ใน Chemical Engineering Transactions ที่ Crossref ไม่มีข้อมูล (ไม่มั่นใจก็ไม่เอา)

`topic` ไม่ถูก embed — ใช้ลงสี node บนกราฟ + gate ว่าต้อง index ใหม่ (ต้องเป็น 1 ใน 6 topic)

`kind: paper` ไม่มีผลความหมาย แต่ลบแล้ว index แยก paper จาก vault note ไม่ได้

4.4 ## Notes ไม่หาย — อะไรรอดเวลา regenerate

body ของทุกการ์ดมีสามก้อนตายตัว ตามด้วย `## Notes`:

****Key findings**** — เจออะไร ใส่เลขจริงให้ครบ
****Thesis relevance**** — เกี่ยวกับการวิจัย
****Full citation**** — reference เต็มแบบ APA

Notes

โน้ตของเราเอง

parser จับสามก้อนด้วย pattern `**label**` — ตรง ๆ เว้นวรรค/สลับ format นิดเดียวจับไม่เจอ ต้องคงรูปแบบเวลาแก้มือ

จุดอันตราย — parser ไม่ error ตอนจับไม่เจอ คั่นสตริงว่างเฉย ๆ พิมพ์ `**Key Finding**` (s หาย)

หรือลิ้ม — → ส่วนนั้นว่างตอน embed แต่ `index` ผ่านปกติ search หาไม่เจอ — บั๊กเจียบแบบเดียวกับ NUL byte บทที่แล้ว

`cards` regenerate การ์ดทั้งชุดจาก JSONL ใช้ตอน JSONL อัปเดต/import เพิ่ม — ทับงานแก้มือได้ถ้าไม่ระวัง อะไรรอดเสมอ กับอะไรอดเพิ่มเมื่อมี `verified:`:

รอดเสมอ ไม่ว่าอะไร	รอดเพิ่ม ถ้ามี <code>verified:</code>
ทุกอย่างได้ <code>## Notes</code>	<code>authors</code>
<code>doi:</code> ที่เคยเติมไว้	<code>title</code>
<code>aka:</code>	<code>journal</code>
<code>authors_upstream:</code>	<code>volume</code>
	<code>issue</code>
	<code>pages</code>

เหตุผล — JSONL ต้นทางผิดตั้งแต่แรก (หัวข้อ 4.6) ปลอ่ย `cards` ทับ field พวกนี้ทุกครั้งเท่ากับ `revert` งาน `verify` ทั้งหมด — มี `verified:` แปลว่า “ห้ามแตะ field พวกนี้แล้ว”

field อื่นที่ไม่อยู่สองคอลัมน์นี้ (`id` `short_title` `quartile` `impact_factor` `topic` `status`

`tags` `kind` + body สามก้อนแรก) ถูกทับด้วย JSONL ทุกครั้งถ้ายังไม่ `verified` แก่ถาวรต้องแก้ที่ JSONL เอง หรือเลิกใช้ `cards`

4.5 store — ไฟล์ธรรมดา 3 ไฟล์ ไม่มี database

`index` เอาการ์ดไป embed แล้วเก็บผลที่ store — ไม่ใช่ database ใช้ไฟล์ธรรมดาสามไฟล์:

ไฟล์	เก็บอะไร
vectors.f32	Float32 เรียงต่อกัน $N \times 1024$ มิติ
meta.jsonl	1 บรรทัด = 1 แถว (id/kind/citekey/title/topic)
manifest.json	{ model, dim, count, updated }

standalone store บนเครื่อง m5 อยู่ที่ `<repo>/citation/store` — `manifest.json` จริง:

```
{
  "model": "ollama:bge-m3",
  "dim": 1024,
  "count": 75,
  "updated": "2026-07-25T11:30:39.102Z"
}
```

75 แถว = 62 paper + 13 vault note (index ด้วย `--vault`) หนักประมาณ 300 KB

`manifest.json` จำชื่อ model เพราะ vector จาก model ต่างกันเทียบกันไม่ได้ — สลับ model ต้อง **index ใหม่ทั้งชุด**

ผ่าน maw ที่เก็บย้ายไปที่ `$MAW_HOME/citation-data/store` แทน — ไฟล์เหมือนเดิม gitignore ลบทิ้งได้ตลอด `index --vault` สร้างใหม่ให้ครบเพราะเป็น derived data ล้วน ๆ

search brute-force cosine ล้วน ๆ ไม่มี ANN index — 75 แถวใช้เวลาประมาณ 0.2 วินาที รวม embed คำค้น ยังไม่ต้องคิด scale จนกว่า corpus จะโตหลักหมื่นแถว

4.6 aka: กับ authors_upstream: — ทำไมไม่มีอะไรถูกลบ

รัน `citation doi` ครั้งแรกถาม Crossref หา DOI เจอว่า corpus ต้นทางผิด 18 จุด ใน 14 การ์ด บางใบผิดหนักถึงใส่ชื่อผู้เขียนคนแรกผิดคนไปเลย

ตัวอย่างจริง — การ์ด `jin2022.md` เดิมชื่อไฟล์ `li2022.md` ใส่ผู้เขียนเป็น Li, R./Hu, Y./Li, H./Zhang, Y. แต่ Crossref ยืนยันผู้เขียนจริงคือ Jin, C. — rename `li2022` → `jin2022` ของเดิมไม่หาย การ์ดใบเดียวกันเก็บสองเวอร์ชันไว้:

```
citekey: jin2022
aka: li2022
authors_upstream:
- "Li, R."
- "Hu, Y."
- "Li, H."
- "Zhang, Y."
```

aka: เก็บ citekey เดิมก่อน rename `authors_upstream`: เก็บรายชื่อที่ JSONL เคยอ้างผิด — สองชั้นของ **Nothing is Deleted**:

- **ชั้นแรก** กันการ์ดผี — `cards` regenerate อ่าน `aka`: รู้ว่า `li2022` ถูก rename เป็น `jin2022` แล้ว ไม่สร้าง `li2022.md` ซ้ำ ไม่มี `aka`: ทุกรอบจะได้การ์ดผีใหม่โผล่เรื่อย ๆ
- **ชั้นสอง** เก็บหลักฐานความผิดพลาด — ไม่ลบของผิดแล้วเขียนทับเงียบ ๆ `authors_upstream` บอก “เคยเชื่อว่าเป็นคนนี้” `authors`: บอก “จริง ๆ คือคนนี้” อยู่ไฟล์เดียวกัน

รายการเต็ม 18 จุด แยกประเภท (ผู้เขียนผิดคน 7 · title แต่งขึ้น 1 · เลขหน้าผิดจากท้าย DOI 7 · journal ผิด 1 · volume สลับเลขหน้า 2) อยู่ที่ `artifacts/citation-audit.md`

สองตัวนี้ไม่คุ้มครองทุกการแก้ไข — เฉพาะ citekey/authors เปลี่ยน การ์ด `she2019` (JSONL แต่ง title เอง citekey ไม่เปลี่ยน) เลยไม่มี field ทั้งสอง ร่องรอยอยู่ที่ `citation-audit.md` /git history เท่านั้น การ์ด `yu2023` (เดิมชื่อ `npjclimate2023` ผู้เขียนไม่เคยผิด ผิดแค่เลขหน้า 363 จากท้าย DOI แทน 41) มี `aka:` แต่ไม่มี `authors_upstream:`

สรุป — `aka:` ตามการเปลี่ยนชื่อไฟล์ `authors_upstream:` ตามการเปลี่ยนเนื้อหา `authors` อีสระจากกัน field อื่น (title, pages, journal, volume) ไม่มี field พิเศษรองรับ ต้องพึ่ง `citation-audit.md` / `git log` — “Nothing is Deleted” ไม่ได้แปลว่า field เดียวเก็บทุกอย่าง แปลว่ามีที่เก็บอยู่เสมอ คนละชั้นตามแต่ทำอะไรเปลี่ยน

บทที่ 5 เรียงคำสั่งทั้ง 8 ตัวตามลำดับใช้จริง เริ่มจาก `status` — โครงสร้างบทนี้คือสิ่งที่ `status` กำลังรายงานให้ฟังทุกครั้งที่เรียก

บทที่ 5: วิธีการใช้งาน — 8 คำสั่ง เรียงตามลำดับที่ใช้จริง

8 คำสั่ง เรียงตามลำดับงานจริง ไม่ใช่ A-Z แต่ละคำสั่ง: คำอธิบายสั้น + คำสั่งจริง + output จริง

ลำดับ	คำสั่ง	ทำอะไร
1	<code>status</code>	เช็คว่าคุณขึ้นส่วนพร้อมไหม — รันก่อนเสมอ
2	<code>cards</code>	JSONL → การ์ด markdown 1 ใบ/paper
3	<code>doi</code>	ถาม Crossref หา DOI + ผู้แต่งจริง
4	<code>bib</code>	การ์ด → <code>.bib</code> พร้อมใช้ใน LaTeX
5	<code>index</code>	embed การ์ดเข้า vector store
6	<code>search</code>	ค้นความหมาย ไม่ใช่ค้นคำ
7	<code>serve</code>	เปิดแผนที่ดาวแบบ interactive
8	<code>graph</code>	render แผนที่ดาวเป็น PNG/SVG

7-8 เป็นคู่ ทำงานบนข้อมูลชุดเดียวกัน — `serve` ลาก-ซูมสด `graph` render เป็นไฟล์นิ่ง

5.1 `status` — รันก่อนเสมอ ไม่มี flag ให้จำ

```
./bin/citation status
```

```
— citation status —
✓ repo root: .../phd-citation-oracle (walk up from script)
✓ 62 paper card(s) in ψ/papers – 61 with a DOI, all citable
✓ corpus present (artifacts/literature_corpus.jsonl)
  – 23814 bytes
✓ store ready – 62 paper(s) + 13 vault note(s) • 1024-dim
  • 300 KB • model ollama:bge-m3
✓ hardware: Apple M5 Max • arm64 • 18 cores • 128 GB unified
✓ embeddings: ollama bge-m3 @ localhost:11434 – no egress
  ↳ bge-m3:latest • 634 MB • 100% GPU • 8192 ctx
✓ arra-oracle-v3 reachable (localhost:47778) – ok [optional]
```

`status` คือประตูของ 7 คำสั่งที่เหลือ — พังจุดนี้จุดเดียว คำสั่งถัดไปพังตามกันเงิบๆ ไม่มี error โผล่ให้เห็น

5.2 `cards` — JSONL → การ์ด (idempotent)

```
./bin/citation cards
```

```
✦ 56 paper card(s) → .../ψ/papers
  created 0 • updated 56 • notes preserved on 2
  upstream citations matched: 56/56
  index: .../ψ/papers/INDEX.md

Next: maw citation index (cards are picked up automatically)
```

ตัวเลข	ที่มา
56	นับจาก corpus JSONL อย่างเดียว
62	ของจริงใน <code>ψ/papers/</code> — อีก 6 มาจาก research ingest

`cards` แต่ละแค่ 56 ใบที่รู้จักจาก JSONL อีก 6 ปล่อยเฉย `INDEX.md` นับจากไต่แรกทอริจริง ได้ 62 เสมอ บรรทัด `Next:` พิมพ์ `maw citation index` คงที่ในโค้ด แม้เรียก ผ่าน `./bin/citation` — ข้อความแนะนำเฉยๆ ไม่ใช่บั๊ก

รันซ้ำได้ตรงๆ — `git diff` ไม่เปลี่ยนแม้แต่ไบต์เดียว เพราะไม่มี timestamp ฝังใน frontmatter

flag เดียว — ระบุ path `.jsonl` อื่นแทน default: `./bin/citation cards path/to/other.jsonl` (แทบไม่ได้ใช้ เพราะ corpus มีไฟล์เดียว)

5.3 `doi` — ถ้าม Crossref (dry run เป็น default)

default คือ dry run ไม่เขียนอะไรจนกว่าจะสั่ง `--write` — Crossref เดาผิดได้ ดูผลก่อนเชื่อดีกว่า เขียนทับแล้วมาแก้

```
./bin/citation doi jarernwong2021
```

```
✦ resolving 1 of 62 card(s) against Crossref
[dry run - pass --write to save]

? jarernwong2021 - no confident match
(best title similarity only 0.76)
  best: [0.76] journal-article 10.3390/atmos14020261
  Health Impact Related to Ambient Particulate ...

resolved 0 • ambiguous 1 • failed 0
```

0.76 ต่ำกว่าเกณฑ์ 0.85 — ปล่อยว่างดีกว่าผูก DOI ผิดใบ นี่คือ ใบเดียวใน 62 ที่ยังไม่มี DOI (ตีพิมพ์ใน Chemical Engineering Transactions ที่ Crossref ไม่ครอบคลุม) ไม่เจาะจงก็ได้ — default คือทุกใบที่ยังไม่มี DOI หรือผู้แต่ง ไม่ครบ ไม่ใช่ทั้ง 62 ใบ

flag	ทำอะไร
<code>--write</code>	เขียนผลจริงลงการ์ด (ไม่ใส่ = dry run เสมอ)
<code>--all</code>	เช็คซ้ำทุกใบ แม้ใบที่มี DOI อยู่แล้ว
<code>--rekey</code>	เปลี่ยนชื่อไฟล์เป็นชื่อจริง (คีย์เก่าเก็บใน <code>aka:</code>)

รันจริงทั้งคลัง — DOI จาก 8/62 เพิ่มขึ้นเป็น 61/62 เจอ error 18 จุด ใน 14 การ์ด ส่วนใหญ่วารสารผิด/หน้าเลขผิด/ผู้แต่งคนแรกผิด (Crossref เชื้อได้ว่า JSONL ต้นทาง) รายละเอียดที่

`artifacts/citation-audit.md`

5.4 `bib` — การ์ด → `artifacts/citation.bib`

```
./bin/citation bib
```

```
• 62 citable entries → .../artifacts/citation.bib
61/62 carry a DOI • 1 without
```

```
Use in LaTeX: \bibliography{artifacts/citation}
then \cite{adong2025}
```

“citable” ไม่ได้แปลว่ามี DOI — แปลว่ามีผู้แต่ง/ชื่อเรื่อง/วารสาร/ปีครบ การ์ดที่ขาดผู้แต่งคอมเมนต์ไว้
ไม่ได้ทิ้ง — withheld บอกว่าขาดอะไร แก้วด้วย citation doi --write แล้วรัน bib ใหม่
flag เดียว — --by-topic จัดกลุ่ม entry ตาม 6 หัวข้อแทนเรียง ตาม citekey มีประโยชน์เวลาเขียน
related work ที่ละหัวข้อ
ตรวจผ่าน bibtex จาก TeX Live 2026 แล้ว — 62 entry เข้า 62 \bibitem ตรงกันเป๊ะ warning
เท่ากับ 0

5.5 index / search — --vault รวม paper กับ note

index แปลงข้อความจากการ์ดเป็นเวกเตอร์ 1024 มิติผ่าน ollama ทำหลัง cards เสร็จ ทำซ้ำเมื่อแก้
title/summary/relevance (แก้แค่ doi: ไม่ต้อง index ใหม่ — ไม่ถูก embed)

```
./bin/citation index --vault
```

```
Loaded 62 paper card(s) from .../ψ/papers
Loaded 13 vault note(s) from ψ/memory/learnings,
    ψ/memory/retrospectives, ψ/memory/resonance,
    ψ/writing/research
Embedded 75 item(s) in batches of 16
Wrote .../.maw/citation-data/store - 75 × 1024-dim
    (300 KB), model ollama:bge-m3

✓ indexed 62 paper(s) across 6 topic(s):
    9 health-policy
    14 low-cost-sensor-calibration
    6 multi-source-fusion-qa
    6 reference-monitoring-bam
    16 satellite-pm25-products
    11 thailand-burning-season
✓ indexed 13 vault note(s) alongside them (searchable
    together)
```

path ของ store แล้วแต่ shell session — มี MAW_HOME export ได้ .maw/citation-data/store
ไม่มีได้ <repo>/.citation/store แทน คนละ session ให้ผลต่างกันได้ — เช็ status เสมอว่า
store อยู่ไหน

--vault = flag ใช้บ่อยสุด — ไม่ใส่ได้แค่ 62 paper ใส่แล้ว search ครอบคลุมโน้ตโนวอลต์ด้วย (retro,
learning, research draft) ค้นครั้งเดียวเจอทั้งคู่
store ไม่ใช่ database — ไฟล์ธรรมดา 3 ไฟล์ (vectors.f32 meta.jsonl manifest.json)
manifest ล็อกชื่อ model ไว้ เปลี่ยน model ต้อง index ใหม่ทั้งชุด เพราะเวกเตอร์คนละ model เทียบ
กันไม่ได้


```
./bin/citation search "low-cost sensor calibration PM2.5" -k 3
```

Top 3 result(s) for "low-cost sensor calibration PM2.5 ...":

```
[0.6481] 📄 paper \cite{bulot2023}
(low-cost-sensor-calibration) Bulot (2023) --
Reference-Grade from Low-Cost Sensors

[0.6287] 📝 note (ψ/writing/research) Environmental
prediction models for PM2.5 ...

[0.5771] 📄 paper \cite{scientificreports2025}
(low-cost-sensor-calibration) ML Mixed Correction
```

อันดับสองไม่ใช่ paper — เป็นโน้ตของเราเอง (📝) นี่คือนี่ที่ `--vault` เปิดทางไว้ ไม่ต้องแยกคันสองรอบ
flag ใ้ช้อย — `-k N` กำหนดจำนวนผลลัพธ์ (default 5), `--json` ต่อ pipe เข้าสคริปต์อื่น

```
./bin/citation search "burning season northern Thailand" \
-k 1 --json
```

```
[
  {
    "similarity": 0.5625,
    "kind": "paper",
    "citekey": "wongnakae2023",
    "title": "Wongnakae et al. (2023) -- RF PM2.5 ... ",
    "topic": "thailand-burning-season",
    "path": "ψ/papers/wongnakae2023.md"
  }
]
```

ไม่มี database ไม่มี ANN index — brute-force cosine ล้วนๆ รัง ผ่าน 75 แถวใช้เวลาราว 0.2 วินาที
(รวม embed คำค้น) ข้อมูลแค่ หลักสิบ ยังไม่ต้องซับซ้อนกว่านี้

5.6 `serve / graph` — ดูกุ่มดาว

ทั้งคู่คำนวณ cosine similarity ระหว่างทุกคู่ paper แล้ววาดเป็น เส้นเชื่อม — graph นี่คือนี่ “ใกล้เคียงกัน
เชิงความหมาย” ไม่ใช่ “ใครอ้างอิงใคร”

`serve` (เรียก `visualize` ก็ได้) เปิด server ที่ localhost แบบ interactive ลาก ชูม hover ดู
รายละเอียดสด default port 5556 ขนพอร์ตเต็ม — ขยับไปพอร์ตถัดไปเองอัตโนมัติ ไม่ล้นด้วย

EADDRINUSE

```
./bin/citation serve --port 5570
```

✦ citation constellation (2D) – <http://localhost:5570>
62 papers · 6 topics · 71 edges (sim > 0.68)
drag to pan · scroll to zoom · click a star for detail

— verbose —

```
layout      t-SNE (PCA-init, deterministic) in 360 ms
similarity   max 0.819 · mean 0.541 over 1891 pairs
edges        71 of 1891 pairs above 0.68 (3.8%)
isolated      15 paper(s) with no edge at this threshold
```

```
most-connected
  8 Nakapan & Hongthong (2022)
  6 Buya et al. (2023)
  6 Choi et al. (2019)
```

“isolated 15” ไม่ใช่บ๊วก — 15 ไปไม่มีเส้นเชื่อมถึงใครที่ threshold 0.68 แค่ว่าเนื้อหาห่างจากบ๊วกอื่น
ลด threshold ถึง จะเห็นเส้นบางๆ โผล่มา

`graph` render ผลเดียวกันเป็นไฟล์หนึ่ง — SVG กับ PNG แบนเอกสาร ได้ตรงๆ ไม่ต้องมี server ค้างไว้

```
./bin/citation graph
```

```
+ citation graph → .../artifacts/citation-network.svg
+ png             → .../artifacts/citation-network.png
62 papers · 6 topics · 1336 edges (cosine > 0.5, max 0.817)
dense – raise threshold for a cleaner graph:
maw citation graph --threshold 0.60
```

threshold default ของ `graph` (0.5) ต่ำกว่า `serve` (0.68) โดยตั้งใจ — ได้ 1336 เส้นทันที เครื่องมือ
เตือน “dense” พร้อม บอกเลขที่ควรลอง

flag	ใช้กับ	ทำอะไร
<code>--threshold N</code>	ทั้งคู่	ปรับความเข้มของเส้น (สูง = น้อยลง แต่มั่นใจกว่า)
<code>--quiet</code>	<code>serve</code>	ตัดส่วน verbose เหลือแค่ banner กับ URL
<code>--html</code>	<code>graph</code>	export หน้า interactive เป็น <code>.html</code> ไม่ต้องรัน server

PNG ต้องมี `sharp` — optional dependency ตัวเดียวในทั้ง repo ไม่มีก็ไม่พัง ได้แค่ SVG พร้อมข้อ
ความ (png skipped – 'sharp' not installed; the SVG above is the figure) ซึ่งคมกว่า
PNG อยู่แล้วในฐานะ vector image

5.7 ลำดับที่ใช้จริงตอนเริ่มจากศูนย์

Copy ทั้งก้อนไปรันได้เลย ไม่ต้องแกะอะไร (ollama pull ใช้เวลา ตามความเร็วเน็ต ที่เหลือเป็นวินาที)

```
git clone \
  https://github.com/Soul-Brews-Studio/phd-citation-oracle
cd phd-citation-oracle

# 1) ต้องมีตัวเดียว – เช็คก่อน
bun --version

# 2) ครบมี – ข้ามได้ถ้าแค่จะทำ .bib อย่างเดียว
brew install ollama
ollama pull bge-m3

# 3) เช็คทุกชิ้นส่วนในที่เดียว
./bin/citation status

# 4) JSONL → การ์ด (idempotent รันซ้ำได้เสมอ)
```

```

./bin/citation cards

# 5) ดูก่อนว่า Crossref จะทำอะไร (dry run)
./bin/citation doi
# พอใจผลแล้วค่อยเขียนจริง + เปลี่ยนคีย์ placeholder
./bin/citation doi --write --rekey

# 6) การ์ด → .bib
./bin/citation bib

# 7) embed เข้า store (--vault พ่วง vault notes ด้วย)
./bin/citation index --vault

# 8) ลองค้นดูสักคำ
./bin/citation search "PM2.5 satellite AOD Thailand"

# 9) ดูแผนที่ดาว — เลือกแบบไฟล์นิ่งหรือแบบ interactive
./bin/citation graph
./bin/citation serve

```

9 ขั้นตอนแต่ 8 คำสั่ง (doi ปรากฏสองครั้ง — dry run ก่อน แล้ว ค่อย `--write`) จากศูนย์ถึงมี `.bib` พร้อมส่งวิทยานิพนธ์ กับ แผนที่ดาวให้มอง ใช้เวลาไม่ถึงห้านาทีบนเครื่องที่มี ollama รัน อยู่แล้ว

บทที่ 6: Index คืออะไร ทำไมต้องทำ

Index = แปลงข้อความแต่ละ paper เป็นเวกเตอร์ 1024 มิติ เก็บไว้ค้นด้วย cosine similarity บทนี้บอกว่า embed อะไร เก็บที่ไหน เร็วแค่ไหน และตอนไหนต้องทำใหม่

6.1 embed คืออะไร

โมเดล bge-m3 — ป้อนข้อความเข้า คายตัวเลข 1024 มิติ ออก ทุก paper ขนาดเท่ากันเป๊ะไม่ว่ายาวแค่ไหน

- multilingual — ไทยกับอังกฤษอยู่ใน vector space เดียวกัน
- ค้นเป็นไทยเจอ paper ภาษาอังกฤษได้เลย ไม่ต้องแปลก่อน

6.2 อะไรถูก embed — ไม่ใช่ทั้งไฟล์

เฉพาะ 3 필ด์ต่อกัน: title + summary + thesis_relevance

```
// The text we embed per paper: title carries the topic,  
// summary + relevance carry the semantic meat that makes  
// similarity meaningful.  
function paperText(p: Paper): string {  
  return [p.title, p.summary, p.thesis_relevance]  
    .filter(Boolean)  
    .join("\n\n");  
}
```

ไม่ถูก embed: doi, volume, pages, journal — metadata ล้วน ไม่มีเนื้อความให้ตีความ

ผลตามมา: เติม doi เข้าการ์ด ไม่ต้อง index ใหม่ เพราะไม่ได้อยู่ใน paperText() ตั้งแต่แรก

vault note ก็ถูก embed ด้วยถ้าเรียก index --vault — ดึงจาก

ψ/memory/{learnings,retrospectives,resonance} และ ψ/writing/research:

```
# index ทั้ง paper และ vault note ในรอบเดียว  
maw citation index --vault
```

ผลลัพธ์ปนกัน แยกด้วยไอคอน 📄 paper / 📝 note:

```
[0.7913] 📄 paper \cite{mahajan2025}  
         (low-cost-sensor-calibration)  
         Mahajan & Helbing (2025)  
[0.8702] 📝 note (ψ/writing/research)  
         Environmental prediction models for PM2.5
```

6.3 cosine similarity

วัดมุมระหว่างเวกเตอร์สองแถว ไม่ใช่ระยะห่าง — ชี้ทางเดียวกันเป๊ะ = 1.0, ตั้งฉาก = 0, ค่าติดลบแทบไม่เจอในทางปฏิบัติ

ทำไมใช้มุมไม่ใช่ระยะทาง: เอกสารยาว/สั้นให้เวกเตอร์ “ยาว” ไม่เท่ากัน มุมตัดปัญหานั้นทิ้ง เหลือแต่ทิศทาง

ก่อนเทียบต้อง **normalise** เวกเตอร์ทั้งสองฝั่งก่อน (หารด้วยความยาวตัวเอง) เพราะ backend ที่ใช้ไม่ normalise มาให้

6.4 ทำไม brute force พอ — ไม่ต้องมี ANN index

Brute force = วนทุกแถวคำนวณ cosine ตรงๆ ไม่มี index พิเศษช่วยข้าม (ต่างจาก ANN ของ Pinecone/FAISS/LanceDB ซึ่งเร็วกว่าแต่ approximate + ต้องมี native dependency)

ตัวเลขจริงที่วัด: corpus 75 แถว คำนวณ 1 ครั้ง (embed query + brute-force cosine ทั้ง 75 แถว) = 0.221 วินาที

```
# ตัวอย่าง search ที่วัดเวลาไว้
time maw citation search "tmux mouse paste"
# real 0m0.221s
```

ขนาด corpus	brute force	ต้อง ANN ใหม่
75 แถว (ตอนนี้)	0.221 วิ	ไม่ต้อง
หลักพัน	คาดว่าเร็วพอ (ยังไม่ได้วัด)	ยังไม่ต้อง
หลักล้าน	ช้าลงชัดเจน	ต้องคิดเรื่อง ANN

corpus PhD ไม่มีทางโตถึงหลักล้าน — 56 paper + note รวม 75 แถว โตขึ้น 10 เท่าก็ยังสบาย แลกกับไม่มี native dependency, ไม่ต้อง compile พิเศษ, แม่นยำ 100% ไม่มี approximate

6.5 store คือไฟล์ธรรมดา ไม่มี database

ไม่มี SQLite ไม่มี Postgres — เก็บเป็นไฟล์ธรรมดา 3 ไฟล์ในโฟลเดอร์เดียว:

```
vectors.f32      # N × 1024 Float32 เรียงต่อกันเป็นแถวดิบ
meta.jsonl       # 1 บรรทัด = 1 paper/note — citekey, title, kind, ...
manifest.json    # { model, dim, count, updated }
```

`vectors.f32` ไม่มี label ปนอยู่เลย — รู้ว่าแถวที่ 5 คือ paper ไหนจาก `meta.jsonl` บรรทัดที่ 5 (ผูกกันด้วยลำดับ ไม่ใช่ key)

`manifest.json` จำ model + dimension + count + updated — กลไกป้องกันความผิดพลาดสำคัญที่สุด เพราะเวกเตอร์คนละ model เทียบกันไม่ได้แม้มิติเท่ากัน สลับ model แล้วเทียบแถวเก่ากับใหม่ปนกัน = ผลลัพธ์ผิดแบบเงิบๆ ไม่มี error เตือน

ขนาดจริงบนดิสก์: ทั้งโฟลเดอร์ 452 KB สำหรับ 75 รายการ — `vectors.f32` กิน 300 KB (75 × 1024 × 4 byte) ที่เหลือคือ text ล้วน

```
# derived data ล้วนๆ — ลบทิ้งได้เลยแล้ว index ใหม่ ไม่มีอะไรพัง
rm -rf artifacts/index/
maw citation index --vault
```

6.6 ตารางตัดสินใจ — ตอนไหนต้อง index ใหม่

ทำอะไร	ต้อง index ใหม่ไหม	เพราะอะไร
เติม doi ในการ์ด	ไม่ต้อง	doi ไม่ถูก embed
เติม volume , pages	ไม่ต้อง	metadata ล้วน ไม่ถูก embed
แก้ title	ต้อง	อยู่ใน paperText() โดยตรง
แก้ summary	ต้อง	อยู่ใน paperText() โดยตรง
แก้ thesis_relevance	ต้อง	อยู่ใน paperText() โดยตรง
เพิ่ม paper ใหม่เข้า corpus	ต้อง	ยังไม่มีแถวของมันใน store
เขียน vault note ใหม่	ต้อง ถ้าอยาก search เจอ	ใช้ index --vault
สลับ embedding model	ต้อง index ใหม่ทั้งชุด	vector คนละ space เทียบกันไม่ได้
ลบโพลเดอร์ store ที่	ต้อง (แต่ไม่มีอะไรพัง)	derived data สร้างใหม่ได้เสมอ

กฎสั้นๆ: แก้ฟิลด์ที่อยู่ใน title / summary / thesis_relevance → ต้อง index ใหม่ แก้ฟิลด์อื่นทั้งหมด → ไม่ต้อง

ข้อจำกัดที่ต้องรู้ — bge-m3 บอก keyword ให้ไม่ได้

Search คืน similarity 0.7913 มา — bge-m3 เป็น black box ตอบไม่ได้ว่า “ใกล้เคียงเพราะคำไหน”

ความหมายกระจายอยู่ทั่ว 1024 มิติ แทะย้อนกลับไม่ได้

หน้า serve จึงโชว์ term overlap ถ่วงน้ำหนักด้วย IDF แทน แปะป้ายชัดว่าเป็นหลักฐานประกอบ ไม่ใช่สาเหตุ — ไม่ใช่ “โมเดลจับคู่เพราะคำนี้” แต่คือ “คำเหล่านี้ปรากฏร่วมกันด้วย ลองดูเป็นเบาะแส”

บทที่ 7: Index แบบ Local — ollama บนเครื่องเรา

คำสั่ง pull model, serve, index -vault บน ollama local — เวลาจริงวัดบน m5 (Apple M5 Max, arm64, 18 cores, 128 GB unified memory), ค่า batch, ตาราง troubleshoot

7.1 ทำไม local เป็น default

corpus ยังไม่ดีพิมพ์ — `thesis_relevance` คือ argument ที่ยังไม่เผยแพร่ ฉะนั้น default ต้องเป็นตัวที่ไม่ออกเน็ต เสมอ

ลำดับ	backend	ต้องมีอะไร	ออกเน็ตไหม
1	ollama (local)	pull bge-m3 + ollama serve	ไม่ออก ← default
2	Cloudflare worker <code>:18787</code>	wrangler login เดิม	ออก ผ่าน worker
3	Cloudflare REST	<code>CF_ACCOUNT_ID</code> + <code>CF_API_TOKEN</code>	ออก ตรง REST

พอ ollama พร้อม ระบบเลือกเองทันที ไม่ต้อง token ไม่ต้อง account ไม่ต้องต่อเน็ต cloud เป็นตัวสำรองเท่านั้น เปิดใช้เมื่อเครื่องไม่มี GPU หรือ ollama ล่มจริงๆ

`thesis_relevance` คือการตีความว่า paper คำ claim ไหนของวิทยานิพนธ์ — ยังไม่เผยแพร่ บาง paper ก็ยังเป็น preprint ส่งออกนอกเครื่องเท่ากับส่งข้อมูล คนอื่นไปด้วย
ระบบไล่บนลงล่าง เจอ ollama ที่พร้อมก็หยุด บังคับ backend ตรงๆ ได้เช่นกัน:

```
CITATION_EMBED=ollama ./bin/citation index --vault
CITATION_EMBED=worker ./bin/citation index --vault
CITATION_EMBED=cf-rest ./bin/citation index --vault
```

7.2 pull model + serve

ดึง model ครั้งเดียว อยู่บนดิสก์ตลอด ไม่ต้องดึงซ้ำ:

```
ollama pull bge-m3
# ครั้งแรกดาวน์โหลดประมาณ 1.2 GB — วัดจากการ pull จริง
# บน m5 ไม่ใช่เลขจากเอกสารของ ollama เอง
```

ต้องมี ollama รันอยู่เบื้องหลัง ถึงจะรับ request embed ได้:

```
ollama serve
# macOS ปกติมี background service รันเองอยู่แล้ว ไม่ต้องสั่งเอง
# ต้องสั่งเองเมื่อ install แบบ manual หรือรันบน CI
```

bge-m3 = ตัวเดียวที่ citation ใช้ (BAAI General Embedding, multilingual, 1024 มิติ) ไทย/อังกฤษ อยู่ vector space เดียวกัน พิมพ์คำถามไทยค้นเจอ paper อังกฤษ ได้ ข้อความที่ embed ต่อ 1 paper คือ title+summary+thesis_relevance เท่านั้น ไม่ใช่ทั้งการ์ด

ไฟล์ตัวอย่าง doi volume pages ไม่ถูก embed — เดิม doi ที่หลังได้ ไม่ต้อง index ใหม่ทั้งชุด

--vault embed vault note จาก `ψ/memory/{learnings,retrospectives,resonance}` และ `ψ/writing/research` ด้วย ติด tag `kind: note` แยกจาก paper — search เจอทั้งสองแบบในผลลัพธ์เดียว

7.3 รันจริง — 75 รายการ 8.4 วินาที

```
./bin/citation index --vault
```

รายการ	ค่าที่วัดจริงบน m5
จำนวนที่ index	75 (62 paper + 13 note)
เวลา index --vault	8.4 วินาที
เวลา search 1 ครั้ง	0.221 วินาที
ขนาด store ทั้งไฟล์เดอร์	452 KB
ขนาด <code>vectors.f32</code>	300 KB
bge-m3 ใน VRAM	634 MB · ctx 8192 · 100% GPU

เงื่อนไข: model ต้อง warm ใน GPU อยู่แล้ว (ครั้งแรกหลังเครื่องว่างนาน เสียเวลาโหลดเข้า VRAM เพิ่มนิดหน่อย) เฉลี่ยวินาทีละ 9 รายการ — เร็วพอสั่ง index ใหม่บ่อยๆ ได้
search คือ embed คำถามแล้วเทียบ cosine กับทั้ง 75 แถวแบบ brute-force ไม่มี ANN index เขียนล้วนด้วย TypeScript — 75 แถวเล็กเกินกว่าต้องใช้โครงสร้าง ซับซ้อน:

```
[0.7913] 📄 paper \cite{mahajan2025}  
  (low-cost-sensor-calibration)  
  Mahajan & Helbing (2025)  
[0.8702] 📝 note (ψ/writing/research)  
  Environmental prediction models for PM2.5
```

📄 คือ paper 📝 คือ note ตัวเลขในวงเล็บคือ cosine similarity ยิ่งใกล้ 1 ยิ่งใกล้เคียงคำถาม ทั้งสองแหล่งอยู่ vector space เดียวกันเพราะใช้ model เดียวกัน

store บนดิสก์ = 3 ไฟล์ธรรมดา ไม่ใช่ database: `vectors.f32` (75 × 1024 float32) `meta.jsonl` (metadata ต่อแถว) `manifest.json` (จำชื่อ model) — สลับ model ต้อง index ใหม่ทั้งชุด ลบไฟล์เดอร์ store ที่ได้เสมอ เพราะเป็น derived data ล้วนๆ

7.4 batch 16 — เสรยทีเดียวหมดแล้วฟัง

embed ไม่ส่งทีละ paper — ส่งเป็นชุดๆ ชุดละ 16 ชุดด้วย env var:

```
CF_EMBED_BATCH=16 # ค่า default
```


ตอน corpus มีแค่ 56 paper ยิงทีเดียวทั้งชุดยังผ่าน พอรวม note ทะลุ 65 รายการ ยิงทีเดียวเริ่มพัง — worker ตอบ error 500 ตรงๆ ไม่ใช่ timeout ค่อยๆ ซ้ำ ไม่ว่าเหตุจะเป็น payload limit หรือ timeout ฝั่ง worker ผลที่เห็น คือ 500 เหมือนกัน

ทางแก้: หั่นชุดละ 16 ส่งทีละชุด รอผลก่อนค่อยส่งชุดถัดไป (75 รายการ = 5 รอบ) เป็น default อยู่แล้ว แต่ถ้า RAM น้อยหรือ backend limit ต่ำกว่า 16 ลด batch ได้ตรงๆ:

```
CF_EMBED_BATCH=8 ./bin/citation index --vault
```

batch เล็กลง = รอบส่งเยอะขึ้น ช้าลงนิดหน่อย แต่แต่ละ request เบาลง ปรับได้ หั่นทีแค่ตั้ง env var ไม่ต้องแก้โค้ด บทเรียน: อย่ายิงทีเดียวหมด — 65 รายการก็พังได้ถ้าไม่หั่น batch เล็กไม่แพง แต่ป้องกันปัญหานี้ได้เกือบทั้งหมด

7.5 troubleshooting

ส่วนใหญ่ที่เจอ คือ ollama ยังไม่รัน หรือรันแล้วแต่ยังไม่ pull bge-m3 :

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
no embedding backend reachable	ollama ไม่ได้รันอยู่ หรือรันแล้วแต่ยังไม่ pull bge-m3	ollama serve แล้ว ollama pull bge-m3
embed error กลางทาง (error 500)	corpus รวมทะลุ ~65 รายการ แล้ว ยังยิง batch เดียวทั้งชุด	ลด CF_EMBED_BATCH=8
nothing indexed yet / search ว่างเปล่า	ยังไม่เคยรัน index --vault เลย store จึงยังไม่มี	รัน ./bin/citation index --vault ก่อน search

เช็คทีละขั้น เริ่มจากถามว่า ollama รันอยู่จริงไหม:

```
curl -s http://localhost:11434/api/tags
# connection refused = ollama ไม่ได้รันอยู่ → ollama serve
# list ว่าง/ไม่มี bge-m3 → ollama pull bge-m3
```

เช็คว่ามี model ขึ้น GPU จริงไหม ด้วย /api/ps :

```
curl -s http://localhost:11434/api/ps
# size = size_vram → resident เต็มเกือบบน GPU (100%, ไม่มี CPU fallback)
```

ollama unload model เองหลังไม่ใช้งาน ~5 นาที (เช็คได้จาก expires_at ใน /api/ps) — index รอบแรกของวันช้ากว่ารอบถัดไปนิดหน่อยเพราะต้อง warm กลับเข้า VRAM ไม่ใช่อาการพัง ยืนยันด้วยตัวเลข size vs size_vram ไม่ใช่ เดาจากความรู้สึก

ถ้าเช็คครบแล้วยัง no embedding backend reachable ลองบังคับด้วย CITATION_EMBED=ollama ก่อน จะเห็น error message ชัดกว่าเดิม อยากเทียบ backend ที่ 2 ตอนดีบั๊ก เปิดคู่กันได้ด้วย wrangler ที่ล็อกอินไว้แล้ว:

```
cd ~/.maw/plugins/cf-embed/worker  
wrangler dev --port 18787
```

ใช้เฉพาะตอนดีบั๊กหรือเครื่องไม่มี GPU จริงๆ — ollama ปกติดี ไม่ต้องเปิด worker คู่ เพราะเท่ากับส่งข้อความออกเน็ตโดยไม่จำเป็น

ปิดท้าย

ไม่ต้องมีเน็ตเลย: `index` `search` `serve` `graph` `bib` `cards` `status` — ที่ต้องมีเน็ตจริงมีแค่ `doi` (อิง `api.crossref.org` ยืนยัน DOI) ยิงทีละ ใบได้ด้วย `citation doi <citekey>` ไม่ต้องยิงทั้งชุด ต่อเน็ตผ่าน tethering คอตัวจำกัดก็ยังสามารถ
ตัวเลขทั้งบนนี้วัดบนเครื่องที่มี GPU พร้อมอยู่แล้ว — m5 เป็น Apple Silicon unified memory ให้ CPU กับ GPU ใช้ RAM ก้อนเดียวกัน ไม่ต้องคัดลอก ข้อมูลข้ามไปมาแบบ discrete GPU ทั่วไป
index บนเครื่องที่ไม่มี GPU เลย ใช้เวลาทีวินาที ยังไม่ได้วัด สักครั้ง ทั้งบนเครื่องที่ปิด GPU ทดสอบ และเครื่อง Intel/AMD ทั่วไป

บทที่ 8: Index ด้วย CPU — เครื่องไม่มี GPU ทำยังไง

เครื่องที่วัดเลขทั้งเล่มนี้คือ m5 — GPU เต็มก่อน ไม่เคยตกไป CPU บทนี้เลยไม่มีเลข “X วินาทีบน CPU” ให้อ้าง มีแต่วิธีวัดเอง เช็คว่าตกไป CPU จริงไหม และลด batch เมื่อ RAM น้อย

8.1 ollama บน CPU — ทำงานได้ ช้ากว่า ไม่ error

ไม่มี GPU (Metal/CUDA/ROCm) ollama สลับไปรัน CPU เจียบๆ ไม่มีข้อความ เตือน —

`ollama pull bge-m3` ทำงานเหมือนเดิม แค่ embed แต่ละ batch ช้ากว่า

- GPU = core เล็กจำนวนมาก ทำ matrix multiplication พร้อมกันหลักพัน — ตรงกับงาน embedding พอดี
 - CPU = core น้อยกว่ามาก งานเรียงคิวแทนที่จะขนาน
 - CPU core เยอะ ≠ เร็วเท่า GPU — หลักสิบยังห่างจากหลักพัน-หมื่นอยู่ดี
 - ตัวเลขจริงขึ้นกับ CPU gen + instruction set (AVX2/NEON) — บอกเป๊ะ ล่วงหน้าไม่ได้ ต้องวัดเอง
- หมายเหตุ: ความเร็ว `ollama pull` วัดจากความเร็วเน็ต ล้วนๆ ไม่เกี่ยว CPU/GPU — คนละปัญหากับความเร็ว index

8.2 เช็คว่าตกไป CPU จริงไหม — size_vram vs size

```
curl -s http://localhost:11434/api/ps | jq .
```

```
{
  "models": [
    {
      "name": "bge-m3:latest",
      "size": 665398500,
      "size_vram": 665398500,
      "context_length": 8192
    }
  ]
}
```

size_vram เทียบ size	แปลว่า
<code>size_vram = size</code>	resident GPU เต็มก่อน — m5 เป็นแบบนี้ (634 MB, 100%)
<code>size_vram < size</code>	บางส่วน GPU บางส่วนตก CPU — VRAM ไม่พอโหลดทั้งก่อน
<code>size_vram = 0</code> หรือไม่มี field	CPU ล้วน ไม่มี GPU ช่วยเลย

`maw citation status` เช็คให้อัตโนมัต:

```
✓ hardware: Apple M5 Max • arm64 • 18 cores • 128 GB — Metal GPU available
✓ embeddings: ollama bge-m3 @ http://localhost:11434 — 1024-dim
  ↳ bge-m3:latest • 634 MB • 100% GPU (fully resident) • 8192 ctx
```

เจอ  partly on CPU หรือ % ต่ำกว่า 100 = เครื่องพึ่ง CPU จริง ไม่ต้อง เดา ไม่ต้องดูพัฒนา

หมายเหตุ: residency เช็คได้หลัง embed ไปแล้วเท่านั้น (ต้องถูกเรียกก่อน ถึงไฟล์ใน /api/ps) — เรียก status ก่อน embed บรรทัดนี้จะหายไปเฉยๆ ไม่ใช่ error

ดูสตรระหว่าง index กำลังรัน:

```
watch -n 2 'curl -s http://localhost:11434/api/ps | jq .'
```

ไม่อยากพึ่ง jq — ollama มี CLI ตรงๆ:

```
ollama ps # คอลัมน์ PROCESSOR บอกสัดส่วน GPU/CPU ตรงๆ
```

คำนวณ % เองแบบที่ maw citation status ทำ:

```
curl -s http://localhost:11434/api/ps \
| jq '.models[] | {name, pct: ((.size_vram // 0) / .size * 100 | round)}'
```

8.3 จับเวลา index ของตัวเอง

```
time maw citation index --vault
```

```
real    0m??.???s
user    0m??.???s
sys     0m0.???s
```

บรรทัด	ความหมาย
real	เวลารอจริง (นาฬิกาผนัง) — เลขที่เทียบข้ามเครื่องได้ (m5 GPU = 8.4s)
user	เวลา CPU รวมทุก core — CPU-only อาจมากกว่า real ถ้าขนานหลาย thread
sys	เวลาคุยกับ OS (อ่านไฟล์ เปิด socket) — ปกติ้น้อยมาก

ข้อควรระวัง: รอบแรก (cold) รวมเวลาโหลดโมเดลเข้า RAM ด้วย รอบสอง (warm) สะท้อนการคำนวณจริงๆ — จัดแยกบรรทัดทั้งสองแบบ

ปิดโปรแกรมหนักๆ ก่อนวัด วัดซ้ำ 3 รอบ เอาค่ากลาง (median) — ครั้งไหน ช้าผิดปกติอาจเป็น thermal throttle หรือโปรแกรมอื่นแทรก:

```
for i in 1 2 3; do
{ time maw citation index --vault; } 2>> index-timing.log
done
```

โน้ตบุ๊กไร้พัดลมอาจเจอ thermal throttling — รอบท้ายช้ากว่ารอบต้นทั้งที่ งานเหมือนเดิม ไม่ใช่ bug พักเครื่องให้เย็นแล้ววัดใหม่

จดผลไว้ที่หาเจอทีหลัง เช่น `"index --vault บนเครื่อง X: cold Ns / warm Ns, size_vram=0%"` —
ใช้เป็น baseline เทียบตอนคอร์ปัสโตขึ้น
อยากเทียบ GPU vs CPU บนเครื่องเดียวกัน: ปิด GPU acceleration ชั่วคราว (ตั้ง environment variable ที่ ollama อ่าน บังคับ CPU-only) แล้ว `time` ซ้ำเทียบกัน

8.4 RAM น้อย — ลด CF_EMBED_BATCH

ตัวขามีสองแบบคนละทาง — ไม่มี GPU (คำนวณซ้ำ แก่ด้วย GPU/backend อื่น) กับ RAM ไม่พอ (swap หนัก ค้างหรือ error 500 แก่ด้วยลด batch) แก้ผิดทางไม่ช่วยอะไร

embed ส่งเป็น batch ค่า default 16 รายการ คมด้วย `CF_EMBED_BATCH` (ชื่อ CF จากยุค backend แรกคือ Cloudflare แต่คุมทุก backend รวม ollama ด้วย):

```
CF_EMBED_BATCH=8 maw citation index --vault
```

batch เล็กลง → tensor ค้างใน RAM พร้อมกันน้อยลง แลกกับรอบมากขึ้น (75 รายการ batch 8 ≈ 10 รอบ แทน ~5) เวลารวมไม่ต่างมาก แต่เสถียรขึ้น

ยังไม่พอ โหลดครั้งต่อครั้ง: `16 → 8 → 4 → 2 → 1` — ที่ 1 คือยิ่งทีละ รายการ แทบไม่ error เรื่อง memory แต่ overhead ต่อรอบสูงสุด ไม่ตั้งเป็น default เพราะกินเวลารวมมากขึ้นถ้า RAM พออยู่แล้ว — ปรับจนกว่า index จะรันจบโดยไม่ error

อาการ	ลองทำ
index ค้างนาน ไม่มี error	ปกติสำหรับ CPU-only — รอ หรือวัดเวลาไว้
500 กลางทาง	ลด <code>CF_EMBED_BATCH</code>
เครื่องแฉ็ก / swap เยอะ	ลด <code>CF_EMBED_BATCH</code> เพิ่ม
no embedding backend reachable	ollama ไม่ได้รันอยู่ — <code>ollama serve</code>

8.5 คอร์ปัส 62 ใบ ไม่ใช่ 62,000 ใบ

คอร์ปัส 62 paper + 13 vault note = 75 รายการ embed แค่ 3 ฟิลด์สั้นๆ ต่อรายการ store ทั้งหมด 452 KB — ต่อให้ CPU ช้ากว่า GPU สิบยี่สิบเท่า ก็จบในไม่กี่นาที ไม่ใช่ชั่วโมง
เครื่องไม่มี GPU ไม่ใช่ข้ออ้างไม่เริ่ม — แครอนานกว่าเดิม

8.6 ตอนไหนควรยอมใช้ cloud แทน

สัญญาณ: `real` นานเกินคาด (รอเป็นสิบนาที่สำหรับแค่ 75 รายการ), ลด `CF_EMBED_BATCH` ถึง 1 แล้ว ยัง error, หรือ RAM แทบไม่พอโหลดโมเดล — ทางออกคือสลับ backend เป็น cloud (Cloudflare Workers AI รันโมเดล เดียวกัน `dcf/baai/bge-m3`):

```
CITATION_EMBED=worker maw citation index --vault
```

ข้อควรคิดก่อนกด: ข้อความที่ส่งไป (title/summary/thesis_relevance ของ วิทยานิพนธ์ที่ยังไม่ตีพิมพ์)
ออกจากเครื่องแล้วเอาคืนไม่ได้ — ไม่ใช่ เหตุผลให้กลัว cloud แต่ให้เลือกอย่างรู้ตัว ไม่ใช่เพราะซีเรียจรอ
รายละเอียดเต็มเรื่อง `worker` vs `cf-rest` อยู่บทที่ 10

บทที่ 9: Index บน M5 Apple Silicon — Metal กับ unified memory

บทนี้ให้: วิธีเช็ค ollama ใช้ GPU จริงหรือแค่อ้าง, คำสั่ง `sysctl` ที่ `citation status` เรียกจริง, เลขจริงจาก m5 (Apple M5 Max, arm64, 18 cores, RAM 128 GB unified memory)

9.1 unified memory ต่างจาก discrete GPU ตรงไหน

discrete GPU (NVIDIA ผ่าน PCIe) มี VRAM แยกจาก RAM หลัก — โหลด model ต้องคัดลอกข้าม PCIe bus ก่อน model ใหญ่กว่า VRAM ต้อง quantize หรือ fallback บางเลเยอร์ไป CPU
Apple Silicon: CPU กับ GPU ใช้ RAM ก้อนเดียวกัน (zero-copy) — ไม่มี ขั้นตอนคัดลอก ชีตจำกัดขนาด model จึงเป็น RAM รวมทั้งเครื่อง ไม่ใช่ VRAM แยกก้อน

ขั้นตอน	discrete GPU (PCIe)	unified memory (Apple Silicon)
โหลด model เข้า GPU	copy RAM → VRAM ข้าม bus	ใช้ RAM ก้อนเดียวกัน
ขีดจำกัดขนาด model	VRAM การ์ดจอ	RAM รวมทั้งเครื่อง (128 GB)
ถ้า model ใหญ่เกิน	swap / fallback CPU บางส่วน	แทบไม่เจอกับ embedding model
ยืนยันว่าอยู่บน GPU จริง	<code>nvidia-smi</code> ดู memory	<code>/api/ps</code> — <code>size_vram</code> vs <code>size</code>

bge-m3 โหลดเข้า memory แค่ 634 MB เทียบ RAM 128 GB แทบไม่รู้สึกรึ้ออะไร — คำถามจริงไม่ใช่ “มี RAM พอไหม” แต่คือ “GPU เห็น model ทั้ง ก้อนจริงไหม หรือมีบางส่วนหลุดไป CPU” เรื่องนี้ตอบด้วยตาเปล่าไม่ได้ ต้องวัด

9.2 ยืนยันว่าใช้ GPU จริง — ไม่ใช่แค่เดา

ollama มี endpoint `/api/ps` คืนรายการ model ที่โหลดอยู่ พร้อม `size` (ขนาด model ทั้งก้อน) กับ `size_vram` (ขนาดบน GPU memory)

```
size_vram = size → resident ทั้งก้อน - 100% GPU
size_vram < size → บางส่วนหลุดไป CPU - ⚠️ partly on CPU
size_vram = 0 → ไม่ได้อยู่บน GPU เลย - CPU ล้วน
```

เรียกตรงๆ ด้วย curl:

```
curl -s http://localhost:11434/api/ps | jq .
```

ผลลัพธ์จริงจากเครื่องนี้ (ตัด field ที่ไม่เกี่ยวออก):

```
{
  "models": [
```

```
{
  "name": "bge-m3:latest",
  "size": 664660868,
  "size_vram": 664660868,
  "context_length": 8192,
  "expires_at": "<เวลาปัจจุบัน + keep_alive>"
}
```

ตารางอ่านค่าแต่ละ field ที่โค้ดของเราใช้จริง:

field	ความหมาย	หมายเหตุ
name	ชื่อ model + tag	เทียบด้วย <code>startsWith(OLLAMA_MODEL)</code> ไม่ใช่ตรงเป๊ะ
size	ขนาด model ทั้งก้อน (byte)	รวมทั้ง CPU + GPU
size_vram	ขนาดที่อยู่บน GPU (Metal) จริง	เท่ากับ <code>size</code> = 100% resident
context_length	context window ที่ negotiate จริง	หน่วย token ไม่ใช่ default
expires_at	เวลาที่จะ unload ถ้าไม่มีการเรียกใช้	นับตาม <code>keep_alive</code> (default ~5 นาที)

`664660868` byte = 634 MB คือค่าจริงที่วัดได้จาก bge-m3 บนเครื่อง นี้ บนเครื่องไม่มี GPU (บที่ที่แล้ว)

`size_vram` จะเป็น 0 หรือไม่มี field นี้เลย — สัญญาว่าตกไป CPU ทั้งหมด

ถ้า `size_vram` มีค่าแต่น้อยกว่า `size` แปลว่า ollama offload บาง เลเยอร์ไป CPU — inference ต้องรอทั้งสองฝั่ง throughput รวมมักแยกว่า กรณีสุดโต่งทั้งคู่ (100% GPU หรือ 100% CPU) เราไม่เจอเคสนี้บน m5 กับ bge-m3 แต่โค้ดเตือนไว้ด้วย ⚠ ถ้าเจอเข้าจริง

9.3 sysctl ที่ status ใช้จริง

`hardwareLine()` ใน `src/index.ts` เรียก `sysctl` ตรงๆ ผ่าน `Bun.spawnSync` — ไม่พึ่ง library ภายนอก:

```
function hardwareLine(): string {
  const bytes = (n: number) =>
    `${(n / 1024 ** 3).toFixed(0)} GB`;
  const sysctl = (key: string) => {
    const r = Bun.spawnSync(["sysctl", "-n", key], {
      stdout: "pipe", stderr: "ignore",
    });
    return new TextDecoder().decode(r.stdout).trim();
  };
  const chip = sysctl("machdep.cpu.brand_string");
  const mem = Number(sysctl("hw.memsize"));
  const cores = sysctl("hw.logicalcpu");
  return (
    `✓ hardware: ${chip} · ${process.arch} · ${cores} cores` +
    ` · ${bytes(mem)} unified memory` +
    ` - Metal GPU available to ollama`
  );
}
```



```
);  
}
```

`-n` คืบแค่ค่า ไม่มีชื่อ key นำหน้า · `stderr: "ignore"` กัน warning หลุดเข้า output · ถ้า `sysctl` ไม่มี (เช่นรันบน Linux) ฟังก์ชันมี เงื่อนไข `process.platform === "darwin"` คลุมอยู่ — ข้ามไปคืบแค่ `platform/arch` เฉยๆ ไม่ error
ส่วน residency ต้องถาม ollama ไม่ใช่ macOS — แยกฟังก์ชัน และ ต้องเรียกหลัง `healthcheck` เท่านั้น:

```
// Ask after embedding, not before: the healthcheck is  
// what loads the model.  
const residency =  
  backend === "ollama" ? await ollamaResidency() : "";
```

เหตุผล: ollama unload model ที่ถ้าไม่มีการเรียกใช้เกิน `keep_alive` (~5 นาที) ถ้าเรียก residency ก่อน embed `/api/ps` จะคืนรายการว่าง — บรรทัด `L` หายไปเสียๆ ไม่ error ไม่เตือน แค่ข้อมูลหายทางแก้: เรียก `checkEmbedBackend()` ให้ embed ก่อนเสมอ แล้วค่อยถาม residency

9.4 เลขจริงบน m5 — Apple M5 Max

รัน `citation status` แล้วได้:

```
✓ hardware: Apple M5 Max · arm64 · 18 cores  
  · 128 GB unified memory — Metal GPU  
    available to ollama  
✓ embeddings: ollama bge-m3 @ http://localhost:11434  
  - 1024-dim  
    L bge-m3:latest · 634 MB · 100% GPU  
      (fully resident — no CPU fallback)  
      · 8192 ctx
```

18 core แยกเป็นสองกลุ่มจริง — `hardwareLine()` เรียกแค่ `hw.logicalcpu` ตัวเดียว แต่เช็คแยกได้:

```
$ sysctl -n hw.perflevel0.logicalcpu hw.perflevel1.logicalcpu  
6  
12
```

`perflevel0` = performance core (6) · `perflevel1` = efficiency core (12) รวม 18 ตรงกับ `hw.logicalcpu` เทียบกับ `hw.perflevel{0,1}.physicalcpu` ได้ค่าเท่ากันเป๊ะ — ไม่มี hyperthreading/SMT บนเครื่องนี้ ยังไม่ได้ต่อ `sysctl` คู่นี้เข้า `hardwareLine()` — วัดเพิ่มเองตอนเขียนบทนี้เท่านั้น

ตัวชี้วัด	ค่าที่วัดได้	ที่มา
ชิป	Apple M5 Max	<code>sysctl machdep.cpu.brand_string</code>
core รวม	18 (6 perf + 12 eff)	<code>sysctl hw.logicalcpu</code>
RAM	128 GB unified	<code>sysctl hw.memsize</code>
bge-m3 ใน memory	634 MB	<code>/api/ps</code> → <code>size</code>
GPU residency	100%	<code>size_vram / size</code>
context	8192	<code>/api/ps</code> → <code>context_length</code>
index 75 รายการ	8.4 วินาที	จับเวลาจริง (model warm)
เฉลี่ยต่อรายการ	~0.112 วินาที	$8.4 \div 75$ (คำนวณ ไม่ใช่วัดแยก)
search 1 ครั้ง	0.221 วินาที	จับเวลาจริง (embed + brute-force cosine)

ไม่ได้แยกวัดว่า embed query กับ brute-force cosine (75 แถว) กิน เวลาเท่าไรต่อชิ้น รู้แค่ตัวเลขรวม — และไม่ได้เทียบ “GPU เร็วกว่า CPU กี่เท่า” (บทที่แล้วตั้งใจไม่แต่งตัวเลข CPU ขึ้นมาเทียบ) สิ่งที พิสูจน์ได้จริงมีแค่: model นี้อยู่บน GPU ครบทั้งก้อน ไม่มี fallback

9.5 ทำไมต้องมีบรรทัดนี้

“GPU-backed” เคยเป็นแค่คำในเอกสาร ไม่เคยปรากฏใน output จริง — claim ที่ไม่มีใครวัดซ้ำได้ในหนึ่งคำสั่ง จะเลิกเป็นความจริงแบบเงิบๆ โดยไม่มีใครรู้ตัว

บทที่ 10: Index ด้วย Cloud — Cloudflare Workers AI

Index ผ่าน Cloudflare แทนเครื่องตัวเอง ใช้ตอนไม่มี ollama หรือไม่มี GPU (เครื่องยืม, CI) — cloud เป็นตัวสำรอง ไม่ใช่ default `detectBackend()` เช็ค ollama ก่อนเสมอ

เลขจริงที่วัดมีตัวเดียว: ollama index 75 รายการ = 8.4 วินาที (บทที่แล้ว) เวลา index ผ่าน worker/cf-rest ยังไม่ได้วัด — เครือข่ายคุมไม่ได้เหมือน GPU ในเครื่อง

10.1 สามทาง: ollama, worker, cf-rest

สองทาง cloud ยังไปหา model เดียวกันคือ `qcf/baai/bge-m3` บน Workers AI ต่างกันแค่ auth กับเส้นทาง request:

ทาง	auth	CITATION_EMBED	egress
ollama	ไม่มี	ollama	ไม่ออกเน็ต
worker <code>:18787</code>	wrangler login เดิม	worker	ออกเน็ต ผ่าน wrangler dev
cf-rest	<code>CF_ACCOUNT_ID</code> + <code>CF_API_TOKEN</code>	cf-rest	ออกเน็ต ตรงไป Cloudflare

บังคับเลือก backend ด้วย `CITATION_EMBED` (ข้าม auto-detect ของ `detectBackend()`):

```
CITATION_EMBED=ollama ./bin/citation index --vault
CITATION_EMBED=worker ./bin/citation index --vault
CITATION_EMBED=cf-rest ./bin/citation index --vault
```

ค่าไม่ถูก validate — พิมพ์ผิด (เช่น `cloud`) ไม่ error ทันที ไหลไปตกที่ worker แบบเงิบๆ เช็คด้วย `citation status` ทุกครั้งหลังตั้งค่านี้ ดูบรรทัด `✓ embeddings: ...` ว่าตรงกับที่ตั้งใจไหม

worker ใช้ AI binding (`[ai]` / `binding = "AI"` ใน `wrangler.toml`) เลยไม่ต้องมี token แยก — wrangler สวมสิทธิ์บัญชีที่ `wrangler login` ไว้ให้ตอน dev server เริ่ม cf-rest ต้องมี token จริง

เหมาะกับสคริปต์ headless เช่น CI ที่ไม่มี `wrangler dev` ค้างให้เรียก

10.2 เปิด worker

worker บันทึกลงใน repo แล้วที่ `ψ/lab/citation/worker/` — ไฟล์เดียว (`worker.js` ~80 บรรทัด) ไม่มี `node_modules`:

```
cd ψ/lab/citation/worker
wrangler dev --port 18787
```

ครั้งแรกต้อง `wrangler login` ก่อนหนึ่งครั้ง (เปิด browser authorize) จากนั้นรัน `wrangler dev` ซ้ำได้เรื่อยๆ ไม่ต้อง login ใหม่

Contract ของ worker มีสาม endpoint เท่านั้น:

```
POST /embed
{ texts: ["a", "b"] } → { data: number[][] }
POST /query-embed
{ text: "a query" } → { data: number[][] }
GET /health
→ { ok: true, model }
```

`/embed` กับ `/query-embed` รับผิดชอบ `body.texts` และ `body.text` — ยิ่ง field ก็ยิ่งทำงานถูกปลั๊กอินฝั่ง citation เรียกผ่าน `CF_EMBED_WORKER_URL` (default `http://localhost:18787` อยู่แล้ว ไม่ต้องตั้งอะไรเพิ่ม)

10.3 ทดสอบจริง

```
curl http://localhost:18787/health
# {"ok": true, "model": "qcf/baai/bge-m3"}

curl -X POST http://localhost:18787/embed \
-H 'content-type: application/json' \
-d '{"texts": ["a", "b"]}'
# {"data": [[0.01, -0.02, ...], [0.03, 0.00, ...]]}
```

ส่ง body ว่าง — worker ไม่ crash ไม่โยน stack trace ตอบ HTTP 400 พร้อมข้อความอ่านรู้เรื่อง:

```
curl -X POST http://localhost:18787/embed \
-H 'content-type: application/json' -d '{}
# {"error": "no non-empty text supplied"}
```

worker ตอบ 500 ถ้า batch ใหญ่เกินไปในคำขอเดียว (56 paper ง่ายๆ ผ่าน, paper+vault note รวมกันไม่ผ่าน) โค้ดจึงตัดเป็นก้อนละ 16 เสมอผ่าน `CF_EMBED_BATCH` (ปรับได้ถ้า corpus ใหญ่มากกว่านี้):

```
const EMBED_BATCH =
  Number(process.env.CF_EMBED_BATCH) || 16;
```

error message บอกดัชนี batch ที่พัง + text ที่ยาวที่สุดในก้อนนั้น ไม่ใช่แค่ “พังที่ไหนสักที่”

10.4 account_id หลายบัญชี

`wrangler.toml` ไม่มี `account_id` โดยเจตนา — repo เป็น public, account ID ไม่ใช่ secret แต่ก็ไม่มีเหตุผลต้องเผยแพร่ ผลข้างเคียง: ถ้าเครื่อง login มากกว่าหนึ่ง account, wrangler หยุดทำงานทันที ด้วยข้อความนี้:

```
More than one account available but unable to
select one in non-interactive mode.
```

เจอเองจริงตอนทดสอบบนเครื่องที่ login สองบัญชี แก้ด้วยการระบุ account ตรงๆ ผ่าน environment variable ก่อนรัน:

```
CLOUDFLARE_ACCOUNT_ID=<your-account-id> \
wrangler dev --port 18787
```

หา id ได้จาก `wrangler whoami` (list ทุก account ที่ login ไว้พร้อม id)

ระวังชื่อ env var สองชุด ไม่ใช่ชุดเดียว: `CLOUDFLARE_ACCOUNT_ID` เป็นของ `wrangler` CLI เอง ใช้ตอนเลือก account ให้ worker ส่วน `CF_ACCOUNT_ID` / `CF_API_TOKEN` (ไม่มี `CLOUDFLARE_` นำหน้า) เป็นของโค้ด `src/index.ts` เองตอนเลือกทาง cf-rest — ตั้งชุดหนึ่งแล้วอีกชุดยังต้องตั้งแยกอยู่ดี

10.5 CORS เปิดกว้าง

worker เปิด CORS ให้ทุก origin (*) เพราะหน้า `citation serve` (บทที่ 12) ยิง query embedding ตรงจาก browser ไม่ผ่าน proxy ฝั่งเรา — worker เองไม่ถือ secret (ไม่มี token, ไม่มี account credential) ความเสี่ยงจริงคือ “โดนเรียกเกิน free tier” ไม่ใช่ข้อมูลรั่ว

10.6 คำเตือน: corpus ที่ยังไม่ตีพิมพ์

ข้อความที่ส่งไป Cloudflare ไม่ใช่ทั้งไฟล์ แต่คือ `title + summary + thesis_relevance` ต่อ paper — เพียงพอจะเป็นเนื้อความงานวิจัยจริง ไม่ใช่ metadata ง่ายๆ worker กับ cf-rest วิ่งผ่านเน็ตจริง ต่างจาก ollama ที่ไม่ออกจากเครื่องเลย — thesis ที่ยังไม่ตีพิมพ์ถ้าหลุดออกไปก่อน เอาคืนไม่ได้

`index --vault` เสี่ยงกว่า index paper ง่ายๆ เพราะดึงโน้ตส่วนตัว (claim, hypothesis) เข้า embed ด้วย ไม่ใช่แค่บทคัดย่อ paper คนอื่นที่ตีพิมพ์แล้ว — ตัดสินใจเองว่าคุ้มไหม ollama ยังเป็น default ที่ถูกเช็กก่อนเสมอในโค้ด

10.7 model tag ไม่เดียวกันเสมอ

สลับ backend แล้ว `manifest.json` เปลี่ยน tag: ollama ได้ `ollama:bge-m3`, worker/cf-rest ได้ `cloudflare:acfb/baai/bge-m3` เดียวกันทั้งคู่ (อิง model เดียวกันบน Cloudflare)

ยังไม่ได้วัดว่า vector จากสอง runtime ตรงกันแค่ไหน — ถือเป็นคนละ index family ห้ามเทียบ cosine ข้ามกัน สลับ backend ต้อง `index --vault` ใหม่ทั้งชุดเสมอ เช็คด้วย `citation status` สองบรรทัด: `embeddings` บอกว่าจะใช้ทางไหนถ้า index ตอนนี้, `store ready` บอกว่าของที่มีอยู่แล้วทำด้วย tag ไหน — สองบรรทัดนี้ไม่ตรงกันแปลว่ายังไม่ได้ index ใหม่หลังสลับ backend

10.8 สลับกลับ local + เก็บกวาด

```
unset CITATION_EMBED
ollama pull bge-m3           # ถ้ายังไม่เคย pull
./bin/citation status        # เช็คว่าเห็น ollama ใหม่
./bin/citation index --vault # index ใหม่ทั้งชุด
```

`wrangler dev` สร้างโฟลเดอร์ `.wrangler/` ขึ้นมาเอง (cache + account identifier) — ต้องใส่

`.wrangler/` เข้า `.gitignore` ก่อน commit เสมอ ไม่มีเหตุผลต้อง commit ของนี้เข้า repo public

บทที่ 11: Cafe Mode — เน็ตไม่มี แบตไม่เยอะ ทำงานยังไง

บทนี้บอกว่า verb ไหนใช้ได้ตอนไม่มีเน็ต verb ไหนต้องมี แล้วให้คำสั่ง เตรียมตัวก่อนออกจากบ้าน ประหยัดแบต และ resolve DOI แบบประหยัด tethering

11.1 ตารางเดี่ยวยจบ

verb	ต้องมีเน็ตไหม	ทำไม
status	ไม่ต้อง	อ่าน sysctl + เช็ค process ในเครื่อง
cards	ไม่ต้อง	อ่าน/เขียน markdown ใน <code>ψ/papers/</code>
index (รวม <code>--vault</code>)	ไม่ต้อง*	embed ผ่าน ollama ที่ localhost
search	ไม่ต้อง*	embed query ผ่าน ollama local เหมือนกัน
serve (= visualize)	ไม่ต้อง	เสิร์ฟ HTML จาก store ในเครื่อง
graph	ไม่ต้อง	render จาก store ในเครื่อง → PNG
bib	ไม่ต้อง	อ่าน card ในเครื่อง → เขียน <code>.bib</code>
doi	ต้อง	ยิงตรงไปที่ <code>api.crossref.org</code>

* = ต้องเลือก backend เป็น ollama (default) ไม่ใช่ `cf-rest` — สลับ ไป Cloudflare REST ตามบทที่ 10 แล้ว `index / search` จะกลายเป็นต้อง มีเน็ตทันที เพราะยิงออกไปหา Cloudflare ตรงๆ

11.2 บังคับ backend ก่อนออกจากบ้าน

`index / search` เลือก backend เอง 3 ชั้น: ollama local ก่อน → ถ้า ไม่เจอ ลอง Cloudflare worker `:18787` (ต้องมี `wrangler dev` รันอยู่ในเครื่อง) → สุดท้ายไปที่ Cloudflare REST (ต้องมี `CF_ACCOUNT_ID` + `CF_API_TOKEN` และเน็ตจริง) ลืม `ollama pull` มา แล้วก็ไม่ได้เปิด `wrangler dev` ไว้ — มันไหลไปจบที่ REST ซึ่งพังทันทีถ้าไม่มีเน็ต (หรือแย่กว่านั้นคือมี tethering นิดหน่อยแล้วมันใช้ไปจริงโดยไม่รู้ตัว) บังคับ backend กันหลุดไปทางอื่น:

```
export CITATION_EMBED=ollama
```

ตั้งไว้ครั้งเดียวใน shell profile — ถ้า ollama ไม่พร้อม มัน error ตรงๆ ทันที ดีกว่าเจิบๆ แล้วไปโดนบิล tethering ที่หลัง

11.3 เตรียมตัวก่อนออกจากบ้าน

Store เป็นไฟล์ธรรมดา 3 ไฟล์ ไม่มี database: `vectors.f32` ($N \times 1024$ Float32), `meta.jsonl`, `manifest.json` รวม 452 KB (75 รายการ: 62 paper + 13 vault note) — `vectors.f32` เอง 300 KB เล็กพอ zip ใส่ USB ได้ ลบทิ้งแล้ว index ใหม่ได้เสมอ เพราะเป็น derived data ไม่ใช่ source of truth

```
# ครั้งแรกดาวน์โหลด model ~1.2 GB - pull ครั้งเดียวพอ
ollama pull bge-m3

# index ทั้ง paper และ vault note - 75 รายการ วัตถุจริง
# (model warm บน GPU แล้ว): 8.4 วินาที
maw citation index --vault
```

Embed เฉพาะ `title + summary + thesis_relevance` ต่อกัน ไม่ใช่ `doi / volume / pages` —

เติม DOI เข้า card ที่หลังไม่ต้อง index ใหม่ เติมได้แม้ตอนไม่มีเน็ต

เช็คก่อนปิดเครื่องด้วย `maw citation status`:

```
— citation status —
✓ store ready - 62 paper(s) + 13 vault note(s) · 1024-dim
  · 300 KB · model bge-m3
✓ hardware: Apple M5 Max · arm64 · 18 cores · 128 GB unified
  memory - Metal GPU available to ollama
✓ embeddings: ollama bge-m3 @ http://localhost:11434 - local,
  no token, no egress - 1024-dim
  ↳ bge-m3:latest · 634 MB · 100% GPU
    (fully resident - no CPU fallback) · 8192 ctx
```

100% GPU = model โหลดเต็มก่อนบน GPU ไม่มีตกไป CPU checklist สุดท้ายก่อนออกจากบ้าน:

```
[ ] ollama pull bge-m3 เสร็จแล้ว - เช็คด้วย `ollama list`
[ ] maw citation index --vault รันผ่าน ไม่มี error
[ ] maw citation status เห็น backend: ollama (ไม่ใช่ cf-rest)
[ ] card ที่ยังไม่มี DOI - resolve ให้ครบตอนนี้
    (doi ต้องมีเน็ตเท่านั้นที่ทำได้จริง)
```

ลิ้มซ้อแรก แล้วถึงร้านค่อยนึกได้ — `index / search` จะ error ทันที เพราะ ollama หา model ไม่เจอ
ในเครื่อง และไม่มีเน็ต pull ให้ก็ไม่ได้ ทางตันทั้งสองทาง เตรียมที่บ้านเท่านั้น ร้านกาแฟไม่มีเน็ตพอให้ pull
1.2 GB แบบสบายใจ

11.4 ประหยัดแบตเตอรี่ — ปิด serve เอง ollama ปิดตัวเอง

`serve` เปิด `Bun.serve()` ค้าง event loop ไว้ตลอด — ไม่จบเองแม้ปิด แท็บเบราว์เซอร์ที่เข้าไปดูแล้ว
ปิดด้วย Ctrl+C ตอนดูจบ ไม่มีเหตุผล ต้องเปิดค้างไว้ทั้งวันถ้าไม่ได้ดูซ้ำ
ollama unload model เองหลังไม่มีการเรียกใช้ประมาณ 5 นาที — ค่า default ของ ollama เอง ไม่ใช่
โค้ดของเรา ปล่อยให้ unload เอง ดีกว่ายิง request ปลอมๆ กันไม่ให้ idle:

```
# ดูว่า model กำลังโหลดอยู่หรือ unload ไปแล้ว - เช็คฟิลด์ expires_at
curl -s http://localhost:11434/api/ps | jq
```

วัตถุจริงบน m5: bge-m3 กิน 634 MB VRAM ที่ context 8192 resident แบบ 100% GPU (ยืนยันด้วย
`size_vram = size` — เท่ากันแปลว่าอยู่ บน GPU ทั้งก่อน ไม่มีตกไป CPU)

11.5 tethering น้อยๆ — doi ที่ละใบ ไม่ใช่ --all

รูปแบบเต็ม: `maw citation doi [--write] [--rekey] [--all] [citekey...]` — ไม่ระบุ citekey และไม่ได้ `--all` มันจะเดาเอง เลือก ทุก card ที่ยังไม่มี DOI หรือ author อัตโนมัติ คำนวณจำนวน request ไม่ได้ — บน tethering เลี้ยงไว้ ระบุ citekey ให้ครบแทน:

```
# dry run - ดูผลก่อน ยังไม่บันทึก
maw citation doi mahajan2025

# พอใจแล้วค่อยเขียนทับ card จริง
maw citation doi mahajan2025 --write
```

dry run กับ `--write` ยัง request ไป Crossref เท่ากัน ต่างกันแค่ ตอนท้ายว่าจะเขียนทับ card หรือไม่ — รู้อยู่แล้วว่าจะบันทึกแน่ ใส่ `--write` ตั้งแต่รอบแรกประหยัดกว่า ไม่ต้องยิงซ้ำสองรอบ ไม่มีเน็ตตอนลอง doi — ไม่ crash ง่ายๆ request ไป `api.crossref.org` จะ timeout แล้ว report error ตรงๆ ว่า resolve ไม่ได้

`serve` ผูก port default 5556 เปลี่ยนได้ด้วย `--port` หรือ env `CITATION_VISUALIZE_PORT` — ลืมปิดรอบก่อนไว้ (Ctrl+C ไม่ทัน) พอเปิด รอบใหม่มันเจอ port ไม่ว่าง แล้วขยับไปหา port ถัดไปเอง อัตโนมัติ:

```
⚠ port 5556 busy (an earlier serve is still running) - using 5557
+ citation constellation (2D) - http://localhost:5557
```

11.6 กลับบ้านแล้วไม่ต้อง sync

ใช้ backend ollama ตลอดวัน ไม่มีอะไรถูกส่งขึ้น cloud เลยสักบรรทัด — card ที่แก้ไขลงดิสก์ทันที ตอนแก้ store update ทันทีตอน `index` รันเสร็จ ไม่มี state ไหนแขวนคอยรอ sync ที่หลัง

ขอยกเว้นเดียว: สลับไป Cloudflare REST ระหว่างวันตามบทที่ 10 (ต้องมี `CF_ACCOUNT_ID` + `CF_API_TOKEN` จริง) — กรณีนั้นข้อมูลออกนอกเครื่อง จริง ต้องดูข้อควรระวังเรื่อง corpus ที่ยังไม่ตีพิมพ์ ตามบทนั้น สำหรับ cafe mode ทางเลือกที่ดีที่สุดคือปล่อยให้ ollama เป็น backend ตลอด ไม่ต้องตัดสินใจอะไรเพิ่ม

บทที่ 12: วิธีการทำ Research — จากคำถามไปเป็น brief

บทนี้ให้: 5 องค์ประกอบที่ brief ต้องมี, template ที่ copy ไป เติมได้จริง, คำสั่งเรียก skill

/gemini-deep-research

```
/gemini-deep-research # ขั้น 1: คำถาม → brief
# → copy brief ไปวางใน Gemini Deep Research / GPT deep research
# → ได้รายงานกลับมา
/research-ingest # ขั้น 2: รายงาน → เช้าคั่งอย่างปลอดภัย
/research-harvest # ขั้น 3: รายงาน → งานจริง
```

ลี skill นี้อยู่ที่ `.claude/skills/` license MIT เปิดอ่านโค้ด ได้เอง ไม่ใช่กล่องดำ

12.1 ทำไม brief สำคัญกว่าคำถาม

deep-research tool ขยายคำถามเป็น plan ก่อนรันจริง — brief ดี กำหนดโครงสร้าง plan ตรงๆ brief คลุม
เครือข่ายได้รายงานกว้างแบบ Wikipedia

12.2 ทำอย่างไรที่ brief ต้องมี

อย่าง	กันปัญหาอะไร
objective ผูกกับการตัดสินใจ	กันรายงานกว้างปลอดภัยไว้มาก่อน แต่ใช้ตัดสินใจไม่ได้
out-of-scope ระบุหัวข้อจริง	กันรายงานเป็นไปเรื่องที่ “เกี่ยวศัพท” แต่ไม่เกี่ยวคำถาม
เกณฑ์ source — ทั้งรับและไม่รับ	กันรายงานปนบล็อก/ข่าวเข้ากับ paper จริง
หัวข้อ output ตั้งชื่อ + เรียงลำดับ	กัน tool จัดโครงสร้างเอง หาแก่นเรื่องไม่เจอ
“สิ่งที่เรารู้แล้ว”	กันรายงานเล่าซ้ำพื้นฐาน (เสียพื้นที่ ~1/3 ของรายงาน)

หัวข้อพึ่งพากันเอง — เขียน objective ให้แม่นก่อน ที่เหลือตาม มายายขึ้น

objective — ก่อน/หลัง

```
# แย่ — ปลายเปิด ไม่ผูกการตัดสินใจ
"ค้นเรื่อง satellite AOD product ในเอเชียตะวันออกเฉียงใต้"

# ดี — ผูกกับการตัดสินใจ มีจุดจบชัดเจน
"เราควรใช้ satellite AOD product ตัวไหนเป็น baseline
เทียบกับโมเดลของเรา — ตัวเลือกมีกี่ตัว แต่ละตัวมีข้อจำกัด
อะไรที่กระทบการตัดสินใจนี้"
```

out-of-scope — ระบุหัวข้อจริง ไม่ใช่ “ไม่เอาเรื่องทั่วไป”

- ไม่ต้องอธิบายหลักการทำงานของ remote sensing ตั้งแต่ต้น
- ไม่ต้องเทียบ sensor ที่ไม่ครอบคลุมเอเชียตะวันออกเฉียงใต้
- ไม่ต้องลงรายละเอียดการประมวลผลสัญญาณระดับ pixel

ยิ่งเจาะจงเท่าไรยิ่งกัน tool ออกจากทางหลงได้มากเท่านั้น — นี่ คือ lever ที่คุ้มที่สุดใน 5 ข้อ เขียนไม่กี่บรรทัด ได้รายงานที่ แคบลงทั้งฉบับ

เกณฑ์ source — สองคอลัมน์เสมอ

รับ:

- peer-reviewed journal article
- official technical report (หน่วยงานรัฐ/องค์กรระหว่างประเทศ)

ไม่รับ:

- preprint ที่ยังไม่ผ่าน peer review
- บล็อกโพสต์ที่ไม่มี citation กำกับ
- เว็บไซต์ข่าวที่เขียนซ้ำ press release โดยไม่อ่าน paper จริง

12.3 ข้อจำกัดของ tool ↔ ข้อบังคับใน brief

ข้อจำกัดของ tool	ข้อบังคับที่ต้องใส่ใน brief
อ่าน full text หลัง paywall ไม่ได้	บังคับ DOI/URL กำกับทุกอ้างอิง
อ่านไฟล์ในเครื่องเราไม่ได้	ห้ามแต่งข้อมูล/ชื่อผู้เขียนขึ้นเอง
รัน code ไม่ได้	flag “unverified” แทนเคา + บอกตรงเมื่อ source ขัดแย้ง

Citation rules:

- ทุกอ้างอิงต้องมี DOI หรือ URL กำกับ
- ห้ามแต่งชื่อผู้เขียน/ชื่อ paper ขึ้นมาเอง
- ยืนยันไม่ได้ → flag “unverified” ห้ามเคา
- source ขัดแย้งกัน → บอกตรงๆ ห้ามเฉลี่ยเป็น consensus ปลอม

สามข้อจำกัดกับสามข้อบังคับคือสาเหตุ-ผลลัพธ์กันตรงๆ ไม่ใช่คน ละเรื่อง อ่าน full text ไม่ได้ → มีแนวโน้มเคาจาก abstract → บังคับ citation ไว้แต่ต้นคือทางเดียวที่ตรวจสอบย้อนกลับได้

12.4 เรียก skill /gemini-deep-research

```
/gemini-deep-research
# เข้ารอบ scoping แบบ interactive ถามทีละข้อ ตามโครง 5 ข้อ:
# 1. คำตอบนี้เอาไปตัดสินใจอะไรต่อ → objective
# 2. มีเรื่องไหนที่รู้อยู่แล้ว ไม่ต้องค้นซ้ำ → what we know
# 3. มีหัวข้อไหนอยากกันออก → out-of-scope
# 4. เกณฑ์ source รับได้/รับไม่ได้ → source quality
# 5. อยากให้จัดหัวข้อ output กี่ส่วน ชื่ออะไร → output sections
#
# ตอบครบ → skill ประกอบทุกคำตอบเป็น brief ฉบับเดียว พร้อม copy
```

skill อยู่ที่ `.claude/skills/gemini-deep-research` — ทำแค่ ครั้งแรกของงาน (คำถาม → brief) ส่วน ingest/harvest อยู่บท ถัดไป

12.5 template brief — copy ไปเติมได้จริง

Objective:

เราควรใช้ satellite AOD product ตัวไหนเป็น baseline เทียบกับโมเดล PM2.5 ของเรา — แต่ละตัวเลือกมีข้อจำกัดอะไรที่กระทบความน่าเชื่อถือของการเปรียบเทียบนี้

Out of scope:

- ไม่ต้องอธิบายหลักการทำงานของ remote sensing ตั้งแต่ต้น
- ไม่ต้องเทียบ sensor ที่ไม่ครอบคลุมพื้นที่เอเชียตะวันออกเฉียงใต้
- ไม่ต้องลงรายละเอียดการประมวลผลสัญญาณระดับ pixel

Source quality:

รับ: peer-reviewed journal article, technical report จาก
หน่วยงานดาวเทียมโดยตรง (NASA, JAXA, KMA)

ไม่รับ: preprint ที่ยังไม่ผ่าน review, บล็อกสรุปข่าว,
conference abstract ที่ไม่มี full paper

Output sections (เรียงตามนี้):

1. ตัวเลือก satellite AOD product ที่ครอบคลุมพื้นที่
2. ข้อจำกัดของแต่ละตัวเลือกที่กระทบการเปรียบเทียบ
3. เกณฑ์ validation ที่แต่ละ product ใช้รายงาน (ถ้ามี)
4. ช่องว่างที่ยังไม่มีใครยืนยันได้จาก public source

What we already know:

- GEMS เป็นดาวเทียม geostationary ครอบคลุมเอเชียตะวันออกเฉียง
- เรามี ground truth จาก BAM monitoring station อยู่แล้ว
- ไม่ต้องอธิบายว่า AOD คืออะไรในเชิงฟิสิกส์

Citation rules:

ทุกอ้างอิงต้องมี DOI หรือ URL กำกับ ห้ามแต่งชื่อผู้เขียน
หรือชื่อ paper ขึ้นมาเอง ถ้ายืนยันไม่ได้ให้ flag ว่า
"unverified" แทนการเดา ถ้า source ชัดแย้งกันเองให้บอก
ตรงๆ ว่าขัดแย้งกันตรงไหน ห้ามเฉลี่ยเป็น consensus ปลอม

โครงนี้เป็นตัวอย่าง เปลี่ยนหัวข้อได้ตามงานจริง แต่ทั้ง 6 ส่วน ต้องอยู่ครบเสมอ (5 จาก 12.2 บวก citation rules จาก 12.3)

12.6 กฎเหล็ก

อ่าน plan ก่อนกด approve ทุกครั้ง ไม่ว่า brief จะดีแค่ไหน กด approve แล้วแก้กลางทางไม่ได้ ต้องรอ
จบก่อนถึงจะรู้ว่าพลาด

บทที่ 13: เอาไปให้ GPT / Gemini อ่าน — ส่งออกยังง

brief จากบทที่แล้ววางใน Gemini Deep Research หรือ GPT ได้เลย แต่ต้องรู้อีกก่อนว่า tool ทำอะไรไม่ได้ — ขอสิ่งที่มันให้ไม่ได้ ก็ได้รายงานผิดหวังกลับมาเหมือนเดิม

13.1 สามอย่างที่ deep-research tool ทำไม่ได้

ทำไม่ได้	เหตุผล	อย่าขอสิ่งนี้
อ่าน paywall	เห็นแค่ title/abstract/DOI ที่ index สาธารณะเปิดให้	รายละเอียดลึกกว่า abstract — ตาราง, สูตร, ค่าใน supplementary material
อ่านไฟล์ในเครื่อง	<code>ψ/papers/*.md</code> , corpus JSONL, vault อยู่นอกสายตา tool แม้ repo ตั้งเป็น public	อย่าคิดว่ามันรู้จัก corpus เราเอง — ต้องยกเนื้อหามาวางในตัว brief เอง
รันโค้ด	ไม่มี interpreter ต่อท้าย มีแต่ข้อ ความล้น	“คำนวณตัวเลขใหม่จากข้อมูลดิบ” (เช่น CCC) — มันจะแต่งตัวเลขที่ดู สมจริงแทนคำตอบว่า “ทำไม่ได้”

ข้อ paywall ไม่ใช่ “อ่านไม่ได้” แบบเห็นหน้าว่างเปล่า — มันเห็นแค่สิ่งที่ index สาธารณะเปิดให้
รายละเอียดลึกกว่า abstract จะเดาจาก pattern ของ paper แนวเดียวกันแทน แต่หน้าตาคำตอบยังคง
มั่นใจเหมือนอ่านจริง
ตัวอย่างข้อสาม — แทนที่จะขอ “คำนวณ CCC ระหว่างค่าเซนเซอร์เรากับ AOD จาก paper นี้” ให้ขอ
“หาว่า literature ใช้ metric ไหนวัด agreement ระหว่างเซนเซอร์ราคาถูกกับ reference monitor และ
threshold ที่ยอมรับได้” — คำถามแบบหลังตอบได้จากสิ่งที่ paper รายงานไว้แล้ว ไม่ต้องคำนวณใหม่
เพราะ tool ไม่มีทางบอกตรงๆ ว่า “ทำไม่ได้”

13.2 ดึง context จาก vault ไปด้วย — `search --json`

tool อ่านไฟล์เราไม่ได้ — ต้องดึง context ออกมาใส่ brief เอง ไม่งั้นรายงานเสียพื้นที่เล่าซ้ำสิ่งที่ corpus
มีอยู่แล้ว

```
./bin/citation search "satellite AOD PM2.5" -k 5 --json
```

output จริง (ย่อเหลือฟิลด์ที่ brief ต้องใช้):

```
[
  {
    "citekey": "bai2021",
    "title": "Bai et al. (2021) -- AOD vs TOA ...",
    "journal": "Aerosol and Air Quality Research (Q2)"
  },
  {
```

```

    "citekey": "o2025",
    "title": "O et al. (2025) -- GEMS AOD to ...",
    "journal": "Atmospheric Measurement Techniques (Q1)"
  }
]

```

`citekey` + `title` + `journal` วางเป็น bullet list ในหัวข้อ “สิ่งที่เรารู้แล้ว” ได้ตรงๆ — `id` ที่ store คืนมาคือ `paper:<citekey>` ไม่ใช่เลขจาก frontmatter (เช่น 2.3.4) อย่าเอาไปเขียนเป็นเลขอ้างอิงใน brief
brief จริงเขียนใน frontmatter ว่า

```
dedupe: 16 satellite/fusion papers already in artifacts/literature_corpus.jsonl
are named in the brief as already-known
```

แล้วเนื้อ brief มีหัวข้อคำไว้พอดี:

```

Already known – do NOT spend research effort
re-finding or summarising these: Kim et al.
(2020) GEMS mission, BAMS · Cho et al. (2024)
first GEMS aerosol results, AMT · Jang et al.
(2025) GEMS AOD validation over mainland SE
Asia, AAQR · O et al. (2025) GEMS AOD to hourly
PM2.5 via ML, AMT · ...

```

สืบทกใบที่ list ไว้คือสืบทกใบที่ deep-research tool จะไม่เสียเวลาหาซ้ำ มันไปหาแต่สิ่งที่เรายังไม่มี (MODIS/MAIAC, VIIRS, TROPOMI, Himawari, MISR) — โฟกัสแคบลงเพราะบอกตรงๆ ว่าอะไรมีอยู่แล้ว ไม่ใช่เพราะขอให้แคบ

`-k` เลือกจำนวนผลลัพธ์ — หัวข้อกว้างต้องตั้งสูง ตั้งต่ำไปจะตัด paper ที่รู้อยู่แล้วออก แล้ว tool ก็เสียเวลาหาซ้ำอยู่ดี

13.3 สามข้อบังคับที่ต้องเขียนในตัว brief

ข้อบังคับ	เคล็ดลับที่ยืนยันว่าจำเป็น
flag claim ที่ยืนยันจาก source ไม่ได้	การ์ด jang2025 เองบันทึกว่า “as reported by that synthesis; not yet checked against the paper’s own text”
DOI ทุกอ้างอิง ห้ามแต่ง	รายงานจริงอ้าง “Chen, X. et al.” คู่ DOI 10.3390/rs11232771 — resolve จริงกลับเจอ She, L. et al. (2019) คนละชื่อคนละ title ทั้งคู่
บอกตรงๆ ตอน source ขัดกัน ห้ามเฉลี่ยเป็นข้อสรุปกลาง	Kim(2020)/Cho(2024) บอก “slight” underestimate แต่ Jang(2025) วัดจริงเจอ slope 0.56 พร้อมทิศกลับที่ AOD 0.5

สองข้อแรกมาจากบรรทัดเดียวกันของ self-check ใน skill `/gemini-deep-research`:

Unverifiable claims must be flagged, and contradictions between sources surfaced rather than averaged into false consensus.

ผลตอนใช้จริง — รายงาน AOD ฉบับเดียวกันตอบกลับมาพร้อมหัวข้อ “Disagreements and Conflicts in the Literature” แยกสองฝั่งไว้ตรงๆ:

Foundational algorithm papers (Kim et al., 2020) and early evaluations (Cho et al., 2024) indicated only "slight" underestimation. However, the most recent operational evaluation by Jang et al. (2025) demonstrates severe, structural underestimation -- slope of just 0.56 -- and this bias reverses at the extreme low end: GEMS overestimates AOD when AERONET is below 0.5, but drastically under-reports during the massive >1.0 AOD spikes typical of March-April in Thailand.

ถ้าเฉลี่ยสองคำอธิบายเป็น “underestimate ปานกลาง” คำตอบจะผิดทั้งสองช่วง — ผิดทิศที่ AOD ต่ำกว่า 0.5 (จริงคือ overestimate) และผิดขนาดที่ AOD สูงกว่า 1.0 ซึ่งเป็นช่วง burning season ที่ thesis ต้องใช้เทียบข้อมูลจริง

รายงานที่เขียนเรียบ น่าเชื่อถือกว่ารายงานที่เต็มไปด้วย “ไม่แน่ใจ” เสมอ — ถ้าไม่สั่งไว้ก่อน tool จะเลือกทางที่กลัวความไม่แน่นอนทิ้งไปเสียๆ

ข้อ DOI มาจากประโยชน์ในหัวข้อ source quality ของ brief:

Cite every entry with authors, year, full title, venue, and a DOI or direct URL. Never invent a DOI or a citation. If you cannot find a DOI, give the publisher URL and say the DOI could not be confirmed.

เคส Chen X. อันตรายเพราะ DOI นั้น “ถูก” — resolve ไปเจอ paper จริง แต่ชื่อคนกับชื่อเรื่องที่แนบมาไม่ตรงกับ paper ที่ DOI ชี้ไป เช็คแค่ resolve ได้ไม่พอ ต้องเช็ค title กับ author แยกกับ Crossref ด้วย (รายละเอียดเต็มบทที่ 14)

brief ที่ทำตามทุกข้อในบทนี้ — บอกข้อจำกัดก่อน ดึง context จาก vault บังคับ flag ที่ยืนยันไม่ได้ บังคับ DOI ห้ามแต่ง บังคับบอกตอน source ชัดกัน — ก็ยังไม่ได้แปลว่ารายงานที่ได้กลับมาจะถูกทั้งหมด brief คุณได้แค่สิ่งที่ ขอ ไม่ได้คุณสิ่งที่ ได้กลับมาจริง — tool ทำตามกฎครบทุกข้อแล้วยังแต่งชื่อ paper ผิดได้อยู่ดี วางไว้ข้าง DOI ที่ถูกต้อง 100% จนตรวจด้วยตาเปล่าไม่เจอ (เคส Chen X. ข้างบน) ของที่ได้จาก Gemini หรือ GPT เชื่อไม่ได้ทั้งหมดจนกว่าจะผ่านการตรวจอีกรอบ — บทที่ 14 พาไปดูว่าตรวจยังไง เก็บ brief ที่เขียนเสร็จไว้ที่

`ψ/writing/prompts/<YYYY-MM-DD>_<slug>_gemini-deep-research.md` — reuse โครงเดิมกับ

หัวข้อถัดไปได้เลย

บทที่ 14: เอากลับมาทำยังไง — ingest แล้วต้อง verify

บทนี้ให้: ลำดับ ingest 5 ขั้นห้ามสลับ, คำสั่ง verify DOI กับ Crossref, ตาราง 5 เคสจริงที่ AI แต่งข้อมูล (จับได้จากรายงานฉบับ เดียว), คำสั่ง `citation doi --write`

14.1 ลำดับ ingest — 5 ขั้นห้ามสลับ

`skill /research-ingest` บังคับลำดับนี้:

ขั้น	ทำอะไร	เครื่องมือ / เกณฑ์
1. เก็บดิบ	เก็บรายงานต้นฉบับ verbatim + provenance	ไฟล์ต้นฉบับแก้ไขไม่ได้ แก้ทั้งหมดไปอยู่ <code>## Review</code> ต่อท้าย
2. verify DOI	เช็คทุก DOI กับ Crossref ก่อนแต่งการ์ด	<code>curl</code> + Crossref API — ห้ามข้าม
3. reconcile	เลือก enrich / create / hold ต่อรายการ	เทียบกับการ์ดเดิมทีละใบ
4. index	embed เข้า store	<code>maw citation index --vault</code>
5. พิสูจน์ search	search หาสิ่งที่เพิ่งเข้าไปให้เจอจริง	กัน frontmatter ผิดจนหลุด index ง่ายๆ

ขั้น 1 สำคัญเพราะพอเริ่มแก้ไฟล์ต้นฉบับ จะแยกไม่ออกอีกว่าอะไรคือ สิ่ง tool พุด กับอะไรคือสิ่งที่เราสรุปเอง

ขั้น 2 คือคำสั่งเดียวที่ห้ามข้าม:

```
curl -s "https://api.crossref.org/works/10.xxxx/yyyy" \
| python3 -c "import json,sys; \
d=json.load(sys.stdin)['message']; \
print(d.get('title',[''])[0]); \
print(d.get('container-title',[''])[0])"
```

ผลแบ่งสามกลุ่ม: **confirmed** (Crossref คีน paper เดียวกัน) **mismatch** (คีนคนละใบ — บันทึกว่าจริงคืออะไร) **not_found** มีแค่ confirmed เท่านั้นที่ผ่านเข้าการ์ดได้ ขั้น 4 index ขั้น 5 คือขั้นที่มักถูกข้าม รายงาน AOD ฉบับนี้จบด้วย:

```
confirmed: 8 DOI ยืนยันกับ Crossref
mismatch: 1 (Chen X./10.3390/rs11232771 → ตัวจริงคือ she2019)
new cards: 6 (nakapan2022 bai2021 choi2019
             meng2015 she2019 chimla2025)
enriched: 2 (jang2025 o2025)
```

ตัวเลขพวกนี้ไม่ใช่ประเด็นหลัก ประเด็นหลักคือ 5 เคสที่เกือบหลุด รอดต่อไปนี้.

14.2 5 เคสที่ AI แต่งข้อมูล — จับได้จากรายงานเดียว

AI แต่งชื่อ paper ได้แบบมีโครงสร้างถูก ไม่ใช่เดามั่ว — ชื่อผู้เขียน สมจริง ชื่อ paper เข้าหัวข้อ วางเคียง DOI จริงเสียด้วยซ้ำ:

เคส	สิ่งที่เกิดขึ้น	ถ้าเชื่อจะเสียหายอะไร
Chen, X. ไม่มีตัวตน	DOI 10.3390/rs11232771 จริง คือ She, Zhang, Wang & Wang (2019) <i>Remote Sensing</i> 11(23):2771 — ผิดทั้งชื่อผู้เขียนและ title พร้อมกัน	เขียน citekey chen2019 ผิด ทั้งที่ paper จริงคือ she2019
title แต่งคู่ DOI ถูก	DOI ตรง Crossref จริง แต่ title ในการดัดยังเป็นชื่อที่ Gemini แต่งไว้ (“Validation of GeoNEX...”) — title similarity เทียบแล้วได้แค่ 0.33	เช็คแค่ DOI แล้วปล่อยผ่าน การดัดผิด อยู่ในคลังได้อีกหลายชั่วโมงโดยไม่มีใครจับ
[Already Catalogued] ชีผัดใบ	Gemini ติดป้าย Bai et al. ว่า catalogued ที่ LGHAP Foundation แล้ว แต่จริงเป็น paper ใบที่ 3 (bai2021) ไม่ใช่ bai2022 / bai2024 ที่มีอยู่	เชื่อ marker แล้วเอา DOI ใหม่เขียนทับการดัดเดิม 2 ใบพังทั้งคู่ พร้อม paper จริงไม่ถูกบันทึกเลย
editorial comment แชนจ์อันดับ	ค้นด้วย title Crossref จัดอันดับ editorial comment (10.5194/amt-2019-46-ec1) ไว้เหนือ paper จริงที่มันวิจารณ์อยู่ — title คล้ายกันมาก ต่างแค่ท้าย DOI มี -ec1	หยิบผลอันดับ 1 มาตรงๆ โดยไม่เช็ค type จะได้ comment แทน paper
subagent ส่ง paper ผี	subagent ที่ซุด timestamp เพิ่มกลับมาพร้อม Huang (2020) กับ Prasad & Singh (2007) — ไม่มีทั้งคู่ใน journal.jsonl ที่เป็น ground truth ของคลัง	ไม่เช็คกับ journal ground truth จะมีการดัดของ paper ที่ไม่มีอยู่จริงเข้าคลัง

บทเรียนตรงไปตรงมา: DOI ที่ถูก ไม่ได้แปลว่า metadata ที่แนบมาถูก ด้วย — title, authors, journal ต้องเทียบกับ Crossref แยกทุกฟิลด์ ไม่ใช่เทียบ DOI อย่างเดียว และวินัยนี้ใช้เท่ากันไม่ว่าคำสั่ง จะมาจาก deep-research tool หรือ subagent ของเราเอง

14.3 verify แยกฟิลด์ — คำสั่งจริง

เช็ค type field ทุกครั้งที่ verify ด้วย title search — รับแค่ journal-article เท่านั้น:


```
curl -s "https://api.crossref.org/works?query.bibliographic=TITLE" \
| python3 -c "import json,sys; \
d=json.load(sys.stdin)['message']['items'][0]; \
print(d.get('type')); \
print(d.get('DOI'))"
# type ต้องเป็น journal-article เท่านั้น
# journal-article-comment / -ec1 → เลื่อนไปผลถัดไป
```

เช็ค paper ที่ subagent ส่งมากับ ground truth ของ journal ในคลัง ก่อนเขียนการ์ด:

```
# journal.jsonl คือรายชื่อ journal จริงในคลัง - ไม่เจอ = ไม่เขียนการ์ด
rg -i "PROCEEDINGS OF THE NATIONAL ACADEMY" journal.jsonl
# ไม่มีผลลัพธ์ → Huang (2020) ไม่ถูกเขียนลงการ์ดใดๆ
```

14.4 citation doi --write — resolve + rekey

```
maw citation doi [--write] [--rekey]
# resolve authors + DOIs against Crossref
# dry run โดย default (ไม่มี --write ไม่เขียนไฟล์)
```

```
citation doi # dry run: ดูว่าจะเปลี่ยนอะไร
citation doi --all --write --rekey # เขียนจริง + rename citekey
# placeholder เป็น author-year

# รู้ DOI อยู่แล้ว แต่ title ที่เก็บไว้ผิด (เคสที่ 2 ข้างบน)
citation doi she2019 --doi 10.3390/rs11232771 \
--trust-doi --write
```

`--trust-doi` คือทางแก้ตรงของเคส “title แต่งคู่ DOI ถูก” — บอก เครื่องมือว่า DOI นี้เชื่อถือได้ ให้ดึง title/authors/journal จริงจาก Crossref มาเขียนทับของเดิมที่แต่งไว้
citekey ต้องตามชื่อผู้เขียนจริงจาก Crossref เสมอ ไม่ใช่ชื่อที่ รายงานอ้างมา — เคส Chen X. ถูกไฟล์เป็น she2019 ไม่ใช่ chen2019

14.5 search ไม่เจอ ≠ ไม่มีอยู่

“ค้นครั้งเดียวหาไม่เจอ” เป็นหลักฐานอ่อนมากกว่าไม่มีอยู่จริง — agentic search หนึ่งรอบพลาดได้ง่าย ก่อนเขียนประโยค “ไม่มีงานวิจัย เรื่องนี้อยู่” ลงวิทยานิพนธ์ ต้องมี search แบบตั้งใจอีกรอบ (เช่น Scopus/Web of Science) ไม่ใช่เชื่อรอบเดียวจากรายงาน deep-research

บทที่ 15: .bib กับบทเรียน 18 ข้อ — ปิดเล่ม

Card 62 ใบ → ไฟล์ `.bib` เดียวที่ `\cite{}` เรียกใช้ได้จริง บทนี้คือคำสั่งสร้าง วิธียืนยันด้วย bibtex ตัวจริง error 18 จุดที่เจอ และที่เราทำพลาดเอง 4 ข้อ

15.1 สร้าง `.bib`

```
maw citation bib          # เขียน artifacts/citation.bib
maw citation bib out.bib  # กำหนดปลายทางเอง
maw citation bib --by-topic # เรียงตาม 6 taproot แทนตัวอักษร
```

อ่านการ์ดทุกใบใน `ψ/papers/` ดึง frontmatter 9 field (author, title, journal, year, volume, number, pages, doi, note) ประกอบ เป็น `article{}` เขียนทับทั้งไฟล์ทุกครั้ง — การ์ดคือความจริง `.bib` generate เข้าได้เสมอ

`--by-topic` คั่นแต่ละกลุ่มด้วย `% — <topic> —` เช็คจำนวน paper ต่อ topic โดยไม่ต้องนับมือไฟล์ขึ้นต้นด้วย header สี่บรรทัดเสมอ — บอกที่มา จำนวน entry และคำเตือนไม่ให้แก้ไขไฟล์นี้ตรงๆ:

```
% Citation Oracle + - generated by "citation
% bib" from ψ/papers/
% 62 citable entries of 62 cards • 61 with a DOI
% The cards are the source of truth: edit those,
% not this file.
```

15.2 “et al.” → `others`

BibTeX ไม่รู้จักคำว่า “et al.” — เห็น string “Taneepanichskul, N., et al.” แล้วเดาว่าทั้งก่อนคือนามสกุลเดียว ทางแก้คือใส่ `others` เป็นชื่อคนสุดท้าย แล้ว style (unsrt, apalike) จะพิมพ์ “et al.” ให้เอง ไม่ต้องเขียนคำนี้อย่าง

```
function bibAuthors(authors: string[]): string {
  const named: string[] = [];
  let truncated = false;
  for (const a of authors) {
    const cleaned = a
      .replace(/,?\s*\bet\s+al\.\s*$/i, "")
      .trim();
    if (cleaned !== a.trim()) truncated = true;
    if (cleaned) named.push(bibEscape(cleaned));
  }
  return [ ...named, ... (truncated ? ["others"] : []) ]
    .join(" and ");
}
```

15.3 ตัวอย่าง entry จริง — `jarernwong2021`

ไม่มี doi ไม่มี note (ยังไม่ verify กับ Crossref) แต่ครบ author/title/journal/year — citable ปกติ:

```

@article{jarernwong2021,
  author = {Jarernwong, K. and Gheewala, S. H. and
    Sampattagul, S.},
  title = {{Health risk map related to particulate
    matter exposure in Chiang Mai, Thailand}},
  journal = {Chemical Engineering Transactions},
  year = {2021},
  volume = {89},
  pages = {229-234}
}

```

entry ที่ไม่มี `note` = การ์ดที่ยังไม่ verify กับ Crossref — ทั้งคลัง 62 ใบ เจอแค่ใบนี้ใบเดียว
 การ์ดที่ขาด field จำเป็นจะไม่หายจากไฟล์ แต่ถูกเขียนเป็น comment ท้ายไฟล์แทน (Rule 1 —
 Nothing is Deleted) ตอนนี้ทั้ง 62 การ์ด ครบ ไม่มีใบไหนอยู่กลุ่ม withheld แล้ว

15.4 ยืนยันด้วย bibtex จริง

```

pdflatex citecheck.tex # รอบ 1: สร้าง .aux
bibtex citecheck # อ่าน .aux + citation.bib → .bbl
pdflatex citecheck.tex # รอบ 2: ดึง .bbl มาประกอบ
grep -c '\\bibitem' citecheck.bbl # นับ = 62
grep -i warning citecheck.blg # ว่วงเปล่า = 0

```

`\nocite{*}` ดึงทุก entry เข้ามาโดยไม่ต้อง cite ทีละตัว ผล: 62 entry ตรงกับ `citation.bib`, 62
`\bibitem` ใน `.bbl`, 0 warning — bibtex เป็นคนละระบบ ไม่รู้จัก field ของเรามา ก่อน ผลตรงกัน
 จึงหนักแน่นกว่าเชื่อ exit code ของสคริปต์ตัวเอง

15.5 error 18 จุด — แยกประเภท

diff การ์ดทุกใบกับสถานะก่อน verify (051014b) แยกเฉพาะ error จริง ไม่นับ reformat/benign —
 เต็มที่ `artifacts/citation-audit.md`

ประเภท error	จำนวน	citekey ตัวอย่าง
ผู้เขียนคนแรกผิดคน	7	buya2025 (เดิม taneepanichskuld2025)
title แต่งขึ้นเอง	1	she2019
เลขหน้าผิด	7	yu2023, ravindra2024, chen2024
journal ผิด	1	thongsame2024
volume ผิด	2	buya2025, chen2024

รวม 18 จุด ใน 14 การ์ด — `chen2024` โดน 3 จุดพร้อมกัน (author, pages, volume) นอกกลุ่มนี้ยังมี 8
 จุด benign completion กับ 2 จุด reformat — ไม่นับเป็น error
 เคสหนักสุด: `buya2023` กับ `wongnakae2023` — citekey เดิมมาจาก “Amnuaylojaroen, T.” ทั้งที่ชื่อนี้
 ไม่อยู่ใน author list จริง ของบทความไหนเลย เอา author ของ paper อื่นมาแปะผิดไป
 ทุกการ์ดที่ rekey เก็บ citekey เดิมไว้ที่ `aka:` — `buya2025` มี `aka: taneepanichskuld2025` ไม่มี
 ใบไหนถูกลบทิ้งจริง

คำเตือน: rekey แล้ว vector store เก่ายังผูก embedding กับ citekey เก่าอยู่ —

`citation doi --write` เตือนต่อท้ายทุกครั้ง ที่ rekey เกิดขึ้น:

```
Re-index so the store matches the new keys:
citation index --vault
```

ไม่งั้น `search` กับ `graph` จะยังพุดถึง citekey ที่ไม่มีการ์ด รองรับแล้ว

15.6 เลขหน้าจาก DOI — บั๊กเจียบที่สุด

4 ใน 7 จุดเลขหน้าผิด มาจากหยิบเลขท้าย DOI มาเขียนเป็นเลขหน้า ตรงๆ — journal born-digital (npj, Scientific Reports) ใช้ “article number” แทนเลขหน้า เลขนั้นฝังอยู่ใน DOI เอง หยิบผิด ตัวในสาย DOI เดียวกันได้ง่าย

citekey	DOI ลงท้าย	เขียนไว้	จริง
yu2023	-00363-w	363	41
ravindra2024	-00833-9	833	326
villarrealmarines2024	-00837-5	837	293
koziel2025	-02069-w	2069	18573

บั๊กนี้เจียบเพราะคุณสมบัติทุกกรณี — 363 ก็ดูเป็นเลขหน้า บทความสั้นปกติ ต่างจาก title ที่แต่งขึ้น ซึ่งเทียบ DOI แล้วได้ similarity ต่ำจนจับได้ (0.33) เลขหน้าไม่มี signal บอกว่าผิดเลย เจอได้ทางเดียวคือ ดึง field เต็มจาก Crossref มาเทียบทีละตัว ไม่ใช่แค่เช็ค ว่า DOI resolve ได้ error กลุ่มนี้ไม่เคยโผล่ตอน `search / graph` เพราะสิ่งที่ถูก embed คือ

`title + summary + thesis_relevance` เท่านั้น เลขหน้า ไม่เคยอยู่ในเวกเตอร์เลย

`citation doi --all` กับ `bibtex` จึงต้อง คู่กับ `search` ต่อไป คนละงานกัน

15.7 ที่เราผิดเอง

ผิดอะไร	บทเรียน
นับ error ตัวเองผิด (c01b2f5 บอก 11, จริง 18)	สร้าง <code>citation-audit.md</code> generate จาก git diff จริงแทนคำกล่าวอ้าง — ไม่ rewrite commit เดิม
บอก Nat ว่า “แก้แล้ว” ทั้งที่ทดสอบผ่าน pipe redirect ไม่เคยแตะ TTY จริง	ทดสอบใน environment ของผู้ใช้ ไม่ใช่ environment ที่สะดวกกับเรา
NUL byte ดิบใน sort key ทำ ripgrep มองไฟล์เป็น binary เกือบเข้าใจผิดว่า <code>sharp</code> ไม่ได้ใช้แล้ว	เขียน escape <code>\u0000</code> แทนตัวอักษรดิบเสมอ
<code>git add -A</code> เผลอเอา <code>.codex/</code> เข้า repo public (Nat จับได้ ไม่ใช่เรา)	review <code>git diff --cached --name-only</code> ก่อน commit ทุกครั้ง ไม่ใช่ <code>-A</code> แบบไม่คิด

นับ error ผิดข้อแรกอันตรายน่ากว่าที่ฟังดู เพราะนับน้อยกว่าความจริง — ทำให้ audit ดูน่าเชื่อถือเกินจริง ไม่ใช่ระแวงเกินจำเป็น

ทั้ง 4 ข้อมีเส้นเดียวกัน — เชื่อกำอธิบายตัวเองมากกว่าหลักฐาน ชำรงนอก มาตรฐานที่ใช้ตรวจการ์ดทุกใบใน
บทนี้ต้องใช้กับงานตัวเอง ด้วย

ต่อจากนี้

citation edges (ใครอ้างอิงใครในเนื้อ paper) ยังไม่มี — กราฟตอนนี้ คือ semantic similarity เท่านั้น
jarernwong2021 ยังไม่มี DOI — journal ของมัน (Chemical Engineering Transactions) ไม่ได้ลง
ทะเบียนกับ Crossref เลย